SMAGUE Valentin

ENSAE 3e année & MVA

Stage de fin d'études

Année scolaire 2025-2026

Learning Self-Representation for Curiosity-driven Deep Reinforcement Learning

INRIA Flowers

Bordeaux, Talence

Maître de stage : Loris Gaven

Du 4 Mai 2026 au 16 Octobre 2026

SMAGUE Valentin

ENSAE 3e année & MVA

Stage de fin d'études

Année scolaire 2025-2026

Learning Self-Representation for Curiosity-driven Deep Reinforcement Learning

INRIA Flowers

Bordeaux, Talence

Maître de stage : Loris Gaven

Du 4 Mai 2026 au 16 Octobre 2026

Remerciements Je tiens à remercier particulièrement mon maître de stage, Loris Gaven, qui a supervisé ce stage. Ses explications ont toujours été claires et ses conseils avisés. J'ai appris énormément sur le domaine, et son aide et sa disponibilité pour discuter et partager des intuitions ont été précieuses. Je souhaite aussi remercier toute l'équipe Flowers, dont l'ambiance chaleureuse a rendu ce stage très agréable à vivre. Je remercie enfin Pierre-Yves Oudeyer, pour avoir rendu ce stage possible, ainsi que le projet `A0191011996` qui m'a permis d'utiliser Jean Zay pour réaliser les expériences présentées dans ce rapport.

Résumé Ce rapport présente une extension de la méthode MAGELLAN dans le contexte du *curriculum learning* automatique pour l'apprentissage par renforcement *goal-conditioned* à grand espace de buts. Cette nouvelle méthode conditionne l'estimation de la compétence de l'agent pour chaque but sur un état de connaissance représenté par un résumé vectoriel de l'historique récent de l'agent, construit par une architecture de type DeepSet. Plusieurs architectures d'agrégation et plusieurs modes de calcul du *learning progress* à partir de cet état de connaissance sont développés et comparés. Les performances de l'agent sont évaluées sur trois environnements développés pour ce stage, conçus pour solliciter des structures différentes de l'espace des buts.

Ce rapport a été rédigé dans le cadre de mon stage de fin d'études commun aux cursus MVA et ingénieur ENSAE. J'ai eu recours à une IA pour fluidifier la forme de certains passages, mais l'intégralité du contenu scientifique et des analyses présentées est mienne.

Table des matières

1 Introduction **3**

2 Cadre théorique et état de l'art **5**
2.1 Formalisme du problème de RL 5
2.2 Deep Reinforcement Learning 7
2.3 Goal-conditioned Reinforcement Learning et Motivation Intrinsèque 9
2.4 Curriculum learning automatique en goal-conditioned Reinforcement Learning 10

3 Conditionner la compétence sur l'état de connaisance ϕ **12**
3.1 Définition de l'état de connaissance 12
3.2 Architectures de l'encodeur ρ 13
3.3 Calcul du learning progress à partir de l'état de connaissance 15
3.4 Entraînement de l'estimateur de compétence 17
3.5 Discussion : problèmes rencontrés et solutions testées 18

4 Environnements **20**
4.1 MiniGrid 20
4.2 Crafter 22
4.3 Fetch 24

5 Résultats **26**
5.1 MiniGrid 26
5.2 Que représente l'état de connaissance ϕ ? 31
5.3 Crafter 33
5.4 Fetch 36

6 Conclusion **39**

A Liste des hyperparamètres **42**

B Mécanismes additionnels non retenus **43**

C Encodage des buts Crafter par arbre de dépendance **44**

D Configurations alternatives de magellan_ks **44**

E Sonde de sensibilité à l'auto-occurrence **45**

F Crafter avec survie **47**

1 Introduction

Ce stage a été réalisé au centre Inria de l'université de Bordeaux, au sein de l'équipe FLOWERS (*FLOWing Epigenetic Robots and Systems*), sous la direction de Loris Gaven. Cette équipe réunit des chercheurs en intelligence artificielle et en sciences cognitives autour de l'apprentissage et du développement, aussi bien chez les machines que chez les humains. Elle s'intéresse à la façon dont les humains apprennent, notamment les enfants, dans le but de reproduire ce processus d'apprentissage chez les machines (Kaplan and Oudeyer, 2007; Oudeyer et al., 2007). Inversement, elle s'appuie sur l'apprentissage automatique pour mieux comprendre l'apprentissage humain, avec des applications dans des domaines tels que les technologies éducatives, la découverte automatisée, ou l'interaction humain-machine.

L'un des travaux majeurs de l'équipe est le développement des agents *autotéliques* (Colas et al., 2019). Cet adjectif, composé du grec *auto* (soi) et *telos* (but), désigne un agent capable de se fixer et d'apprendre à maîtriser ses propres buts. L'une des questions principales à laquelle l'équipe tente de répondre est la suivante : comment un agent placé dans un environnement ouvert, chargé de maîtriser une multitude de buts de natures et de difficultés différentes, doit-il choisir lui-même les buts sur lesquels concentrer sa pratique, de façon à optimiser son apprentissage global ? La difficulté vient à la fois de la rareté de la récompense et du grand nombre de buts à maîtriser. En effet, lorsque l'espace des buts est grand, un agent explorant au hasard a une probabilité très faible d'atteindre un but donné, et donc de recevoir un signal d'apprentissage. Ce problème est encore amplifié par l'hétérogénéité des buts eux-mêmes, puisque certains sont atteignables presque par hasard tandis que d'autres requièrent des séquences d'actions longues et spécifiques. Dans les deux cas, continuer à répartir uniformément l'effort d'entraînement entre tous les buts gaspille l'essentiel du budget d'interaction sur des buts déjà maîtrisés ou, à l'inverse, encore hors de portée.

Le *curriculum learning* automatique répond à ce problème en laissant l'agent choisir lui-même, à chaque épisode, le but sur lequel s'entraîner, plutôt que de fixer un ordre a priori. Le critère développé et utilisé par l'équipe pour guider ce choix est le *learning progress* (Oudeyer et al., 2007; Portelas et al., 2020). On priorise les buts sur lesquels la compétence de l'agent évolue le plus vite plutôt que les buts déjà stables, maîtrisés ou hors de portée. Plusieurs approches ont été développées dans l'équipe. D'abord l'Absolute Learning Progress (ALP) (Colas et al., 2019), qui suit directement, pour chaque but, un historique de réussite récent pour estimer la compétence de l'agent sur chaque but, et calculer son progrès par la différence entre la compétence actuelle et la compétence passée. Ensuite, l'approche MAGELLAN (Gaven et al., 2025) a été développée. Elle consiste en une estimation de la compétence de l'agent sur chaque but grâce à un réseau de neurones entraîné en continu, ce qui permet de généraliser son estimation à des buts proches de ceux déjà rencontrés. Cette estimation n'est cependant pas théoriquement valide, car la fonction de compétence cible change au cours de l'entraînement, puisque la politique de l'agent progresse, on cherche donc à approximer une cible mouvante.

Ce constat est le point de départ de ce stage, qui tente de répondre à la question suivante : peut-on concevoir un estimateur de compétence dont la cible de prédiction reste stable dans le temps, tout en conservant la capacité de MAGELLAN à généraliser son estimation à des buts peu ou jamais pratiqués directement ?

Pour répondre à cette question, ce stage propose de conditionner l'estimateur de compétence sur un état de connaissance, conçu comme un résumé de l'historique récent de l'agent construit à partir des buts pratiqués et de leurs résultats. Pour construire cet état de connaissance à partir de l'historique de l'agent, ce rapport développe et compare plusieurs architectures d'agrégation (par moyenne, par attention, par auto-attention, et par réseau récurrent), suivant le principe des architectures DeepSet, ainsi que plusieurs mécanismes pour en dériver un signal de *learning progress* exploitable par le curriculum. Il présente également trois environnements de test, construits spécifiquement pour ce stage afin de disposer de structures d'espace de buts différentes, sur lesquels une étude empirique est menée sur le cluster de calcul Jean Zay, comparant cette approche à MAGELLAN et aux méthodes de curriculum plus simples qui l'ont précédée, ainsi qu'une étude de la représentation de l'état de connaissance elle-même.

La Section 2 pose le formalisme du *reinforcement learning* et présente les travaux antérieurs sur le *curriculum learning* automatique, dont MAGELLAN, sur lesquels s'appuie ce stage. La Section 3 présente la méthode développée pendant ce stage, la définition de l'état de connaissance, les architectures testées pour le construire, les façons d'en dériver un signal de *learning progress*, et comment entraîner ces architectures. La Section 4 décrit les trois environnements développés pour évaluer cette méthode. La Section 5 présente les résultats obtenus. Enfin, la Section 6 conclut ce rapport.

2 Cadre théorique et état de l'art

Cette section pose le cadre théorique sur lequel s'appuie le reste du rapport. On part du formalisme standard du Reinforcement Learning (Section 2.1), avant de voir comment les méthodes classiques s'adaptent au *Deep* RL et en particulier à PPO, l'algorithme utilisé dans ce stage (Section 2.2). On introduit ensuite le cadre *goal-conditioned*, dans lequel un agent doit apprendre à atteindre une multitude de buts plutôt qu'une tâche unique, ainsi que les approches de motivation intrinsèque qui en découlent (Section 2.3). Enfin, on se concentre sur le *curriculum learning* automatique, qui motive directement la contribution de ce stage (Section 2.4).

2.1 Formalisme du problème de RL

En Reinforcement Learning, un agent choisit des actions dans un ensemble d'actions $\mathcal{A}$ pour évoluer dans un environnement constitué de différents états dans un ensemble $\mathcal{S}$. A chaque instant t, l'agent choisit une action $A_t \in \mathcal{A}$ pour passer de son état S_t à l'état S_{t+1}. Cette transition est déterminée par la fonction de transition $\mathcal{T}$ qui caractérise la distribution du nouvel état S_{t+1} en fonction de l'état actuel S_t et de l'action prise A_t, elle est notée $\forall (s, s', a) \in \mathcal{S} \times \mathcal{A}$,

$$\mathcal{T}(s'|s, a) = \mathbb{P}(S_{t+1} = s'|S_t = s, A_t = a).$$

Après que l'agent ait effectué A_t, il reçoit de l'environnement la récompense $R_{t+1} = R(S_t, S_{t+1}, A_t)$, où R est la fonction de récompense non stochastique. Initialement, S_0 est généré selon une distribution ρ_0. Ce processus, noté $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{T}, R, \rho_0, \gamma\}$, est appelé *Markov Decision Process* (MDP) et est représenté sur la Figure 1. γ est introduit en suivant.

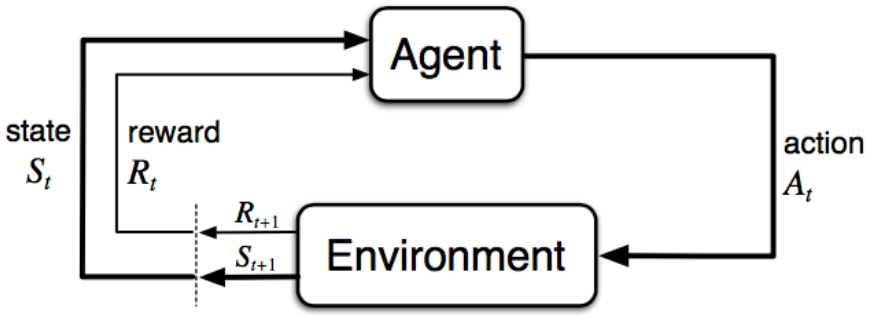


FIGURE 1 – Interaction agent-environnement dans un MDP : l'agent choisit une action A_t sachant l'état S_t, reçoit en retour l'état suivant S_{t+1} et la récompense R_{t+1}.

Une trajectoire est donc une succession d'état, d'action et de récompense :

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots$$

La trajectoire complète, appelée épisode, s'étend au maximum jusqu'à un horizon qui dépend de l'environnement. Il peut être fini comme infini. $\mathcal{T}$ et R définissent le modèle. Dans le cadre de ce stage, nous ne considérons que des cas où le modèle est inconnu de l'agent, c'est le cadre *model-free*.

On définit alors le gain à l'instant t comme la somme des récompenses futures actualisées :

$$G_t := \sum_{k=0}^{\infty} \gamma^k R_{t+1+k}$$

(la somme s'arrête à l'horizon si celui-ci est fini), où $\gamma \in [0, 1]$ est le **facteur d'actualisation** introduit précédemment. Il pondère l'importance accordée aux récompenses futures par rapport aux récompenses immédiates, et garantit la convergence de cette somme à horizon infini dès que $\gamma < 1$.

Pour maximiser l'espérance de son gain, l'agent choisit ses actions au moyen d'une **politique**. On considère ici des politiques stationnaires et stochastiques, c'est-à-dire une application

$$\pi : \mathcal{S} \times \mathcal{A} \to [0, 1], \qquad \pi(a|s) := \mathbb{P}(A_t = a \mid S_t = s),$$

telle que $\forall s \in \mathcal{S}, \sum_{a \in \mathcal{A}} \pi(a|s) = 1$, et qui ne dépend pas de t. A chaque instant t, sachant $S_t = s$, l'action A_t est donc tirée selon $\pi(\cdot|s)$.

Fixer une politique π induit, avec $\mathcal{T}$ et la loi initiale ρ_0, une loi de probabilité bien définie sur les trajectoires $(S_0, A_0, R_1, S_1, A_1, \dots)$. On note $\mathbb{E}_\pi[\cdot]$ l'espérance prise sous cette loi jointe. C'est sous cette espérance que sont définies les fonctions **state-value** et **action-value**, qui mesurent la qualité de π à partir d'un état (ou d'un état et d'une action) donnés :

$$\forall (s, a) \in \mathcal{S} \times \mathcal{A}, \quad \begin{cases} V_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] \\ Q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] \end{cases}$$

Le but de l'agent est de trouver une politique qui maximise cette valeur, autant que possible depuis n'importe quel état de départ. Autrement dit, on cherche, pour chaque état $s \in \mathcal{S}$, une politique optimale au sens de V_π :

$$\forall s \in \mathcal{S}, \quad \pi_s^* \in \operatorname*{argmax}_\pi \ V_\pi(s). \tag{1}$$

A priori, rien n'assure que la politique optimale π_s^* obtenue pour un état s donné soit également optimale pour un autre état $s' \neq s$. Sous des conditions de régularité usuelles, typiquement, $\mathcal{S}$ et $\mathcal{A}$ finis, R bornée, et $\gamma \in [0, 1)$ (ou $\gamma = 1$ avec horizon fini), Puterman (1994) montre qu'il existe en réalité une politique déterministe π^* simultanément optimale pour tous les états. Le cadre considéré dans ce stage vérifie naturellement ces conditions.

On note alors π^* une telle politique optimale, V^* sa fonction de valeur, définie par $\forall s \in \mathcal{S}, V^*(s) := V_{\pi^*}(s) = \max_\pi V_\pi(s)$, et $Q^*(s, a) := Q_{\pi^*}(s, a)$ la fonction action-valeur optimale associée. Enfin, la politique

optimale π^* se déduit directement de Q^* par

$$\pi^*(s) \in \underset{a \in \mathcal{A}}{\operatorname{argmax}}\ Q^*(s,a), \qquad \forall s \in \mathcal{S}. \tag{2}$$

Q^* vérifie également l'**équation de Bellman d'optimalité**, qui découle directement de la relation de récurrence $G_t = R_{t+1} + \gamma G_{t+1}$ vérifiée par le retour :

$$Q^*(s,a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q^*(S_{t+1}, a') \;\Big|\; S_t = s, A_t = a\Big], \qquad \forall (s,a) \in \mathcal{S} \times \mathcal{A}. \tag{3}$$

Cette équation caractérise Q^* de façon récursive, sans référence explicite à une politique. Autrement dit, connaître Q^* suffit à obtenir une politique optimale, sans avoir à résoudre explicitement le problème de maximisation sur l'espace des politiques : c'est ce principe qui fonde les méthodes dites **value-based** en apprentissage par renforcement, où l'on cherche à estimer Q^* directement, plutôt que d'optimiser π dans son espace. Cette approche n'est cependant pas la seule envisageable, comme le montre la section suivante.

2.2 Deep Reinforcement Learning

En pratique, lorsque $\mathcal{S}$ ou $\mathcal{A}$ sont trop grands voire continus pour représenter Q^*, V^* ou π explicitement, on les approxime par des réseaux de neurones. On note alors $\hat{Q}_\phi$, $\hat{V}_\phi$ ou π_θ ces approximations, ϕ et θ désignant les paramètres des réseaux correspondants. Une distinction majeure entre les méthodes présentées ci-après concerne l'origine des données utilisées pour l'apprentissage. On dit qu'une méthode est **on-policy** lorsque les transitions $(S_t, A_t, R_{t+1}, S_{t+1})$ utilisées pour mettre à jour les paramètres doivent avoir été collectées exclusivement par la politique courante elle-même. À l'inverse, une méthode est **off-policy** lorsque les données peuvent avoir été générées par une politique différente de celle en cours d'apprentissage comme une politique passée par exemple, ce qui permet de réutiliser un buffer de transitions déjà collectées.

L'approche value-based la plus directe, le **Q-learning**, apprend $\hat{Q}_\phi \approx Q^*$ par bootstrap sur l'équation de Bellman d'optimalité, à partir de transitions collectées sous n'importe quelle politique de comportement : c'est une méthode **off-policy**, ce qui en fait un des algorithmes les plus efficaces en nombre d'interactions avec l'environnement. Elle présente cependant deux limites importantes. D'abord, le argmax sur $\mathcal{A}$ nécessaire pour dériver π^* de $\hat{Q}_\phi$ (cf. équation (2)) devient coûteux lorsque $\mathcal{A}$ est de très grande dimension, voire intractable si$\mathcal{A}$ est continu. Aussi, la politique qui en résulte est nécessairement déterministe, ce qui est mal adapté aux environnements nécessitant une politique stochastique pour explorer par exemple.

Une alternative consiste à revenir au problème d'optimisation (1) en le rendant tractable en restreignant l'optimisation à une famille paramétrique de politiques $\{\pi_\theta\}_{\theta \in \Theta}$, et on optimise directement les paramètres θ par montée de gradient sur la fonction objectif

$$J(\theta) := \mathbb{E}_{S_0 \sim \rho_0}\big[V_{\pi_\theta}(S_0)\big],$$

en s'appuyant typiquement sur le *policy gradient theorem* (Sutton et al., 2000), qui donne une expression de

$\nabla_\theta J(\theta)$ ne nécessitant pas de différentier $\mathcal{T}$ qui est inconnue dans notre cadre model-free. Cette expression est évaluée sous une espérance prise selon les trajectoires générées par π_θ. Donc, les méthodes de policy gradient sont **on-policy**. En contrepartie, elles permettent de représenter des politiques stochastiques et de traiter des espaces d'actions continus, au prix d'une optimisation non convexe en θ et d'une variance souvent élevée des estimateurs de gradient.

Pour réduire cette variance, les méthodes **actor-critic** combinent les deux approches précédentes, en utilisant un acteur π_θ (la politique, optimisée comme ci-dessus) et un critique $\hat{Q}_\phi$ ou $\hat{V}_\phi$ (une estimation apprise comme dans les méthodes value-based). **Proximal Policy Optimization (PPO)**, introduit par Schulman et al. (2017) et utilisé dans ce stage, fait partie de cette famille de méthodes. Son fonctionnement est décrit en suivant.

À chaque itération, un batch de trajectoires est collecté sous $\pi_{\theta_{\text{old}}}$, puis réutilisé pour plusieurs pas de descente de gradient successifs, avant de recollecter de nouvelles données sous la politique mise à jour. Cette réutilisation à court terme n'est valide que grâce au ratio d'importance sampling

$$r_t(\theta) := \frac{\pi_\theta(A_t|S_t)}{\pi_{\theta_{\text{old}}}(A_t|S_t)},$$

qui corrige le décalage entre la politique ayant produit les données et celle en cours d'optimisation. On note également $\hat{A}_t \approx \hat{Q}_\phi(S_t, A_t) - \hat{V}_\phi(S_t)$, qui mesure si l'action A_t est meilleure ou pire que la moyenne des actions possibles dans l'état S_t sous la politique courante. En pratique, il est estimé par GAE (Schulman et al., 2016).

L'objectif de l'acteur est la fonction *clipped surrogate* :

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big], \tag{4}$$

où $\epsilon > 0$ (typiquement $\epsilon = 0.2$) fixe l'amplitude de déviation tolérée entre π_θ et $\pi_{\theta_{\text{old}}}$. L'opérateur clip tronque $r_t(\theta)$ dans $[1-\epsilon, 1+\epsilon]$, empêchant l'objectif de récompenser une mise à jour qui pousserait $r_t(\theta)$ trop loin de 1. Ce mécanisme répond au défaut pratique des méthodes qui l'ont précédé : une mise à jour de θ trop agressive éloignait trop la nouvelle politique $\pi_{\theta_{\text{new}}}$ de $\pi_{\theta_{\text{old}}}$ ayant servi à collecter les données, sans retour possible.

En parallèle, le critique $\hat{V}_\phi$ est entraîné par régression, en minimisant la *value loss* :

$$L^{\text{VF}}(\phi) = \mathbb{E}_t\Big[\big(\hat{V}_\phi(S_t) - \hat{G}_t\big)^2\Big]. \tag{5}$$

Cet entraînement est ce qui permet à $\hat{V}_\phi$ de rester une ligne de base fiable pour le calcul de $\hat{A}_t$ utilisé ci-dessus.

Enfin, pour éviter une convergence prématurée de π_θ vers une politique quasi-déterministe qui limiterait l'exploration, on ajoute un terme d'entropie, qui encourage π_θ à rester stochastique :

$$L^{\text{ENT}}(\theta) = \mathbb{E}_t\big[\mathcal{H}(\pi_\theta(\cdot|S_t))\big], \qquad \text{où } \mathcal{H}(\pi_\theta(\cdot|s)) := -\sum_{a\in\mathcal{A}} \pi_\theta(a|s)\log\pi_\theta(a|s) \tag{6}$$

est l'entropie de Shannon de la distribution $\pi_\theta(\cdot|s)$, maximale lorsque $\pi_\theta(\cdot|s)$ est uniforme sur $\mathcal{A}$.

La fonction de perte totale optimisée par PPO combine ces trois termes :

$$L(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_1 L^{\text{VF}}(\phi) - c_2 L^{\text{ENT}}(\theta), \tag{7}$$

où $c_1, c_2 > 0$ sont des coefficients pondérant respectivement la value loss et le bonus d'entropie.

PPO ainsi défini fournit l'algorithme d'optimisation utilisé tout au long de ce stage, indépendamment de l'environnement ou de la tâche considérés. Dans la sous-section suivante, nous précisons donc le cadre applicatif dans lequel il est employé : un agent devant apprendre, non pas une tâche unique, mais une multitude de buts différents, chacun pouvant s'avérer plus ou moins difficile à atteindre.

2.3 Goal-conditioned Reinforcement Learning et Motivation Intrinsèque

En goal-conditioned reinforcement learning, la fonction de récompense R dépend de la réalisation d'un but $g \in \mathcal{G} \subset \mathcal{P}(\mathcal{S})$, et s'écrit alors $R(S_t, A_t|g)$. Par exemple, on peut considérer, et c'est le cas dans ce stage, la fonction de récompense binaire $R(S_t, A_t|g) = \mathbf{1}_{\{S_t \in g\}}$. Ainsi, on ne donne une récompense que lorsque l'agent atteint certains états ciblés. La politique dépend alors également du but et s'écrit $\pi(a|s, g) = \mathbb{P}(A_t = a|S_t = s, g)$. $\mathcal{T}$ reste indépendante du but. Dans tout ce qui suit, on se place dans ce cadre.

Lorsque $|\mathcal{S}|$ est grand, la récompense peut devenir très rare, et peut ne plus être suffisante pour que les algorithmes classiques convergent vers la politique optimale. En effet, l'agent agit essentiellement au hasard tant qu'il n'a jamais atteint un but car il n'a pas de gradient exploitable pour progresser, et ici, il a une probabilité extrêmement faible de recevoir un signal de récompense non nul. Ce problème est d'autant plus important lorsque $\mathcal{G}$ est grand : les buts faciles peuvent être atteints très rapidement par hasard, tandis que les difficiles restent hors de portée pendant une grande partie de l'entraînement.

Pour pallier ce problème, une partie de la littérature propose d'enrichir l'agent d'un objectif intrinsèque, indépendant de la récompense extrinsèque R, qui le pousse à explorer son environnement même en l'absence de signal externe. On distingue notamment trois grandes familles d'approches. La première, dite **novelty-seeking**, encourage l'agent à visiter des états qu'il n'a pas ou peu rencontrés jusque-là, en associant à chaque état une mesure de nouveauté. Par exemple, Pathak et al. (2017) entraîne un modèle inverse qui apprend d'abord une représentation $\varphi(s)$ en prédisant l'action A_t à partir de la paire $(\varphi(S_t), \varphi(S_{t+1}))$, ce qui force φ à ne retenir que les aspects de l'état réellement affectés par les actions de l'agent. Un modèle direct prédit ensuite $\varphi(S_{t+1})$ à partir de $(\varphi(S_t), A_t)$, et c'est l'erreur de cette prédiction qui sert de récompense intrinsèque, poussant l'agent vers les transitions les moins bien prédites, donc les moins explorées.

La deuxième famille repose sur la **réduction d'incertitude**. Ici, l'agent maintient une estimation de son incertitude épistémique sur la dynamique de l'environnement et reçoit une récompense intrinsèque d'autant plus élevée que l'action considérée réduit cette incertitude. (Sorg et al., 2010) utilise, par exemple, une distribution a posteriori sur les paramètres d'un modèle de $\mathcal{T}$. Ainsi, l'agent est incité à explorer en priorité les zones où son modèle du monde est le moins fiable.

Enfin, la troisième famille cherche à maximiser le **learning progress**. Elle diffère des deux précédentes en ce qu'elle ne récompense pas directement la nouveauté ou l'incertitude, mais la vitesse à laquelle l'agent améliore sa compétence sur une tâche donnée (Kaplan and Oudeyer, 2007). Plutôt que de répartir uniformément l'entraînement sur tous les buts de $\mathcal{G}$, l'agent peut suivre l'évolution de son taux de réussite par but, et concentrer sa pratique sur ceux où ce taux progresse le plus rapidement. Cela permet de prioriser les buts où il y a un réel apprentissage, et donc de passer petit à petit des buts faciles atteignables presque par hasard, aux but difficiles requierant des successions d'actions spécifiques et pour lesquels la récompense se fait rare. C'est cette dernière approche, développée dans la section suivante, qui est retenue dans ce stage.

2.4 Curriculum learning automatique en goal-conditioned Reinforcement Learning

Comme évoqué précédemment, lorsque $\mathcal{G}$ contient un grand nombre de buts, répartir uniformément les épisodes d'entraînement sur tous les buts est inefficace : certains buts sont déjà maîtrisés et n'apportent plus de signal d'apprentissage, d'autres restent hors de portée de l'agent et ne produisent presque jamais de récompense. Le **curriculum learning automatique** consiste à faire choisir à l'agent lui-même, à chaque épisode, quel but pratiquer, en fonction de son propre état d'apprentissage, plutôt que de suivre un ordre ou une distribution de buts fixés a priori.

Colas et al. (2019) (CURIOUS) proposent d'estimer, pour chaque but $g \in \mathcal{G}$, une mesure de *learning progress absolu* (ALP), calculée à partir de l'historique des épisodes pratiqués sur g. Pour cela, on maintient pour chaque but une estimation de la performance récente de l'agent notée $C_g(t)$ à l'instant t, par exemple par son taux de réussite, et on définit

$$\mathrm{ALP}_g(t) := \left|C_g(t) - C_g(t-\tau)\right|, \tag{8}$$

où τ est une fenêtre temporelle comme un nombre fixé d'épisodes déjà pratiqués sur g par exemple. La valeur absolue dans (8) permet de capturer aussi bien une amélioration qu'une dégradation de la performance, qui peut arriver lorsque l'agent oublie une compétence en se concentrant sur d'autres buts. Dans les deux cas, un ALP non nul indique un signal d'apprentissage disponible.

L'agent échantillonne alors le but sur lequel s'entraîner pour un épisode soit avec une probabilité proportionnelle à $\mathrm{ALP}_g(t)$, soit via une distribution softmax sur les ALP, ce qui concentre l'entraînement sur les buts où la performance évolue le plus, qu'il s'agisse de buts en cours de maîtrise ou de buts nécessitant un petit rafraîchissement. On couple cette probabilité avec un échantillonnage uniforme occasionnel pour continuer à évaluer l'ALP de buts peu pratiqués, dont l'estimation serait sinon trop bruitée.

L'ALP telle que définie par (8) suppose implicitement que l'historique pertinent pour estimer le progrès de l'agent sur g se résume à quelques épisodes récents pratiqués spécifiquement sur g. Cette hypothèse devient limitante lorsque $\mathcal{G}$ est très grand ou structuré (id est quand des buts proches partagent des compétences proches), auquel cas l'expérience acquise sur des buts voisins pourrait informer utilement l'estimation du progrès sur g, même en l'absence d'épisodes pratiqués directement sur g. Gaven et al. (2025) (MAGELLAN) proposent d'estimer, pour chaque but g, la compétence de l'agent via un estimateur $C_{\theta_t}(g) \approx \mathbb{P}_{\pi_t}(g)$ de

la probabilité de succès de la politique courante π_t sur g, entraîné par minimisation de l'entropie croisée binaire sur un buffer des épisodes les plus récents. Le learning progress est ensuite obtenu, comme pour l'ALP, en comparant deux évaluations de cet estimateur à des instants différents :

$$\widehat{\mathrm{ALP}}_{\pi_t}(g) = \left| C_{\theta_t}(g) - C_{\theta_{t-N}}(g) \right|, \tag{9}$$

où θ_{t-N} désigne les paramètres de l'estimateur conservés N mises à jour plus tôt. Contrairement à l'ALP de CURIOUS, $C_{\theta_t}(g)$ ne dépend que du but g ce qui permet à MAGELLAN de généraliser son estimation de compétence à des buts encore peu ou jamais pratiqués, dès lors qu'ils sont sémantiquement proches de buts déjà rencontrés.

Le temps t n'intervient donc dans $C_{\theta_t}(g)$ qu'à travers les paramètres θ_t, mis à jour au fil de l'entraînement. Deux évaluations de la compétence de l'agent sur un même but g, à deux instants distincts, ne sont donc comparables qu'indirectement, via l'évolution des poids θ_t, et rien ne garantit que deux épisodes d'entraînement où l'agent se retrouverait, par ailleurs, dans un état de connaissance similaire produisent une estimation de compétence cohérente, puisque cet état de connaissance n'est jamais représenté explicitement dans l'estimateur.

Ce stage propose d'introduire cette représentation manquante en conditionnant directement l'estimateur de compétence sur un **état de connaissance** (*knowledge state*) $\phi_t \in \Phi$, censé résumer ce que l'agent sait et a expérimenté jusqu'à l'instant t, en plus du but g. L'idée est que la compétence de l'agent sur g dépend de ce qu'il a appris jusque-là, et non du temps écoulé en tant que tel, donc conditionner explicitement sur ϕ_t permettrait ainsi à l'estimateur de généraliser sa prédiction à partir de l'état de connaissance de l'agent, plutôt que de devoir réapprendre implicitement cette dépendance au fil des mises à jour de θ_t. La section suivante traite donc de comment construire et représenter un tel état de connaissance à partir de l'historique de l'agent.

3 Conditionner la compétence sur l'état de connaisance ϕ

Cette section introduit la solution développée lors du stage, l'approche MAGELLAN avec le conditionnement sur l'état de connaissance qu'on appelle aussi *knowledge state* et qu'on note ϕ. Elle explique la définition formelle du *knowledge state*, comment l'implémenter et l'entraîner, et comment choisir le goal sur lequel s'entraîner à partir de l'estimation de compétence.

3.1 Définition de l'état de connaissance

Le but est donc de modéliser un estimateur de la fonction de compétence

$$\begin{aligned} C : \mathcal{G} \times \Phi &\to [0, 1] \\ (g, \phi) &\mapsto \mathbb{P}_\pi(g \mid \phi) \end{aligned}$$

où Φ est un espace de représentation de l'état de connaissance à définir, et $\mathbb{P}_\pi(g \mid \phi)$ désigne la probabilité de succès de la politique courante π sur le but g, sachant l'état de connaissance ϕ. Ainsi, ϕ sert de contexte pour mesurer la compétence de n'importe quel but, et l'estimateur de compétence peut généraliser grâce à ce contexte en plus de la représentation des buts. La différence avec la fonction de compétence introduite pour MAGELLAN (Section 2.4) est donc le conditionnement sur ϕ plutôt que sur l'instant t. Dans MAGELLAN, $C_{\theta_t}(g)$ est une cible mouvante. Avec cette nouvelle modélisation, la relation entre (g, ϕ) et le succès devrait rester stationnaire au cours de l'entraînement, c'est-à-dire que si l'agent se retrouve dans le même état de connaissance ϕ à deux instants différents, il devrait produire la même compétence pour un but g donné.

MAGELLAN utilisait un MLP pour estimer la fonction de compétence $C_{\theta_t}(g) \approx \mathbb{P}_{\pi_t}(g)$, et utilisait un LLM pour encoder le but. J'ai décidé de garder cette implémentation, en concaténant simplement le vecteur d'entrée du but à un vecteur représentant ϕ, mais ce vecteur de but n'est plus encodé par un LLM. Son encodage dépend désormais de l'environnement considéré (voir Section 4). La question principale reste donc de comment construire ce vecteur d'état de connaissance ϕ. L'idée développée ici est la suivante : pour représenter l'état de connaissance à l'instant t, on conserve les K derniers résultats d'entraînement (g, r) (couple but/résultat binaire succès-échec), stockés dans une fenêtre glissante w_t implémentée par un deque. Bien sûr, on n'utilise pas cette fenêtre telle quelle sinon le vecteur obtenu serait de taille trop grande, et ne résumerait que les résultats des buts présents dans la fenêtre, alors qu'on souhaite avoir un résumé global, qui fonctionne également pour des buts absents de w_t. On utilise donc une transformation ρ qui prend la fenêtre w_t en entrée et produit $\phi_t = \rho(w_t)$. Le choix de ρ est discuté en Section 3.2. Figure 2 récapitule l'implémentation.

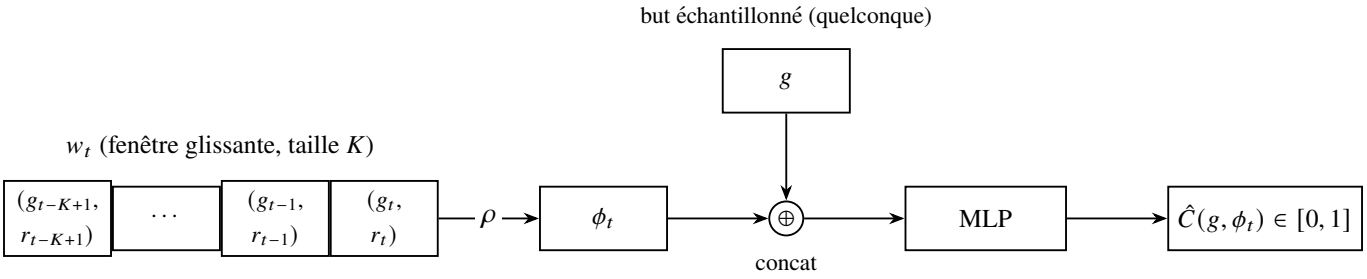


FIGURE 2 – Schéma de l'estimateur de compétence conditionné sur l'état de connaissance. La fenêtre glissante w_t (les K derniers couples (but, résultat)) est résumée par ρ en un vecteur ϕ_t, concaténé au but échantillonné g – quelconque, pas nécessairement présent dans w_t – avant d'être passé au MLP qui produit l'estimation de compétence $\hat{C}(g, \phi_t)$.

3.2 Architectures de l'encodeur ρ

Comme introduit en Section 3.1, ρ transforme la fenêtre $w_t = \big((g_{t-K+1}, r_{t-K+1}), \ldots, (g_t, r_t)\big)$ en un vecteur $\phi_t \in \Phi$. Dans toutes les architectures que j'ai développées, ρ se décompose en trois étapes communes suivant le principe des architectures DeepSet (Zaheer et al., 2017), dont seule la deuxième change d'une architecture à l'autre :

1. **Encodage par item.** Chaque couple (g_i, r_i) de la fenêtre est d'abord vu comme un vecteur brut de dimension $d_g + 1$, résultant de la concaténation du vecteur de but $g_i \in \mathbb{R}^{d_g}$ et du résultat binaire $r_i \in \{0, 1\}$). Ce vecteur est ensuite transformé par un petit MLP ψ, partagé entre tous les items de la fenêtre :
$$\psi(g_i, r_i) = \mathrm{ReLU}\Big(W_2\,\mathrm{ReLU}\big(W_1[g_i; r_i] + b_1\big) + b_2\Big) \in \mathbb{R}^{d_h},$$
où d_h est un hyperparamètre représentant la dimension cachée, et $[g_i; r_i]$ désigne la concaténation. Le fait que cet encodeur soit partagé par tous les items permet à l'agrégation qui suit d'être, selon les architectures, indifférente à l'ordre des items.
2. **Agrégation.** Les K vecteurs $\psi(g_i, r_i)$ sont combinés en un seul vecteur $\mathrm{pool}(w_t) \in \mathbb{R}^{d_p}$ (d_p dépendant de l'architecture). C'est cette étape qui distingue les architectures présentées ci-après, et qui fait l'objet de cette sous-section.
3. **Projection finale.** Un second petit MLP τ, commun à toutes les architectures, ramène le vecteur agrégé dans l'espace Φ :
$$\phi_t = \tau\big(\mathrm{pool}(w_t)\big) = W_4\,\mathrm{ReLU}\big(W_3\,\mathrm{pool}(w_t) + b_3\big) + b_4 \;\in\; \Phi = \mathbb{R}^{d_\phi}.$$

Ainsi, pour chaque architecture décrite ci-dessous, $\phi_t = \tau\big(\mathrm{pool}(\psi(g_{t-K+1}, r_{t-K+1}), \ldots, \psi(g_t, r_t))\big)$.
L'agrégation la plus simple est une moyenne non pondérée des K représentations par item,

$$\mathrm{pool}_{\mathrm{mean}}(w_t) = \frac{1}{K}\sum_{i=1}^{K}\psi(g_i, r_i) \;\in\; \mathbb{R}^{d_h}, \qquad \phi_t = \tau\big(\mathrm{pool}_{\mathrm{mean}}(w_t)\big).$$

Toute la capacité du modèle réside dans ψ et τ, pas dans la façon dont les items sont combinés. Par construction, $\text{pool}_{\text{mean}}$ est invariante par permutation, et chaque item pèse exactement $1/K$, donc un seul événement surprenant, par exemple un succès inattendu sur un but jusque-là toujours raté, n'a qu'une influence diluée en $1/K$ sur ϕ_t, ce qui rend cette architecture robuste au bruit mais lente à réagir à une information isolée, même informative.

Au lieu d'une moyenne uniforme, on peut ajouter une pondération apprise. Chaque représentation $\psi(g_i, r_i)$ est alors projetée en une clé $k_i = W_k\,\psi(g_i, r_i)$ et une valeur $v_i = W_v\,\psi(g_i, r_i)$ au sens de Vaswani et al. (2017), comparées à un vecteur requête q, un unique paramètre du modèle, partagé par tous les buts requêtés, qui ne dépend ni de g ni de t. L'agrégation se calcule alors ainsi :

$$\alpha_i = \text{softmax}_i\left(\frac{q \cdot k_i}{\sqrt{d_h}}\right), \qquad \text{pool}_{\text{att}}(w_t) = \sum_{i=1}^{K} \alpha_i\, v_i \;\in\; \mathbb{R}^{d_h}, \qquad \phi_t = \tau\big(\text{pool}_{\text{att}}(w_t)\big).$$

Cette méthode d'agrégation est toujours invariante par permutation car les scores α_i ne dépendent que du contenu de chaque item, et non de sa position dans w_t, mais l'agrégation peut désormais sur-pondérer certains items plutôt que de les traiter tous également, la pondération q étant elle-même apprise pendant l'entraînement du MLP de compétence.

Les méthodes d'agrégation *mean* et *attention* comparent chaque item indépendamment des autres. En effet, la représentation $\psi(g_i, r_i)$ d'un item ne dépend jamais des autres items de la fenêtre. Dans la troisième méthode d'agrégation développée, *transformer*, les items interagissent d'abord entre eux via plusieurs couches d'auto-attention, toujours au sens de (Vaswani et al., 2017), avant d'être réduites par une simple moyenne. Cette méthode se résume en

$$\big(\widetilde{\psi}(g_1, r_1), \ldots, \widetilde{\psi}(g_K, r_K)\big) = \text{SelfAttention}\big(\psi(g_1, r_1), \ldots, \psi(g_K, r_K)\big),$$

$$\text{pool}_{\text{transf}}(w_t) = \frac{1}{K}\sum_{i=1}^{K} \widetilde{\psi}(g_i, r_i) \;\in\; \mathbb{R}^{d_h}, \qquad \phi_t = \tau\big(\text{pool}_{\text{transf}}(w_t)\big).$$

C'est donc la même agrégation finale que *mean*, mais elle est appliquée à des représentations qui ont déjà eu l'occasion de s'influencer mutuellement. Par exemple, la représentation d'un échec peut être modulée par la présence d'un succès récent sur un but proche, avant même l'étape d'agrégation.

La quatrième et dernière méthode d'agrégation utilise un LSTM (Hochreiter and Schmidhuber, 1997) pour traiter w_t comme une véritable séquence chronologique. $\text{pool}_{\text{lstm}}(w_t)$ est défini comme le dernier état caché h_K de ce réseau :

$$h_i = \text{LSTM}\big(\psi(g_i, r_i), h_{i-1}\big), \qquad \text{pool}_{\text{lstm}}(w_t) = h_K \;\in\; \mathbb{R}^{d_h}, \qquad \phi_t = \tau\big(\text{pool}_{\text{lstm}}(w_t)\big).$$

Ce choix n'est donc pas invariant par permutation. La dernière tentative ajoutée ($i = K$, la plus récente) a mécaniquement une influence disproportionnée sur l'état final, le réseau récurrent la traitant en dernier. C'est un compromis délibéré, on perd la robustesse au bruit apportée par l'invariance par permutation, en

échange de la capacité à exploiter l'ordre chronologique réel des items comme information supplémentaire.

3.3 Calcul du learning progress à partir de l'état de connaissance

Maintenant que ϕ est défini et qu'on peut calculer $C(g, \phi)$, il faut décider de quel but pratiquer à chaque épisode. Pour cela, on suit le même principe que MAGELLAN (Section 2.4) : comparer deux évaluations de la compétence à des instants différents, la différence signalant qu'un but est progrès ou en déclin. La différence avec MAGELLAN est que la comparaison porte maintenant sur des sauvegardes successives de ϕ, et non plus sur des sauvegardes des paramètres θ_t du réseau. On utilise un buffer $\mathcal{N}$ de taille fixe N, implémenté par un deque, contenant les N derniers états de connaissance observés à intervalles réguliers $\phi^{(0)}, \ldots, \phi^{(N-1)}$, avec $\phi^{(N-1)} := \phi_t$ le plus récent. On utilise ce buffer $\mathcal{N}$ pour mesurer le *learning progress* de différentes façons que j'ai développées lors de ce stage.

La mesure **endpoints** compare simplement le ϕ courant au plus ancien du buffer, de manière analogue à MAGELLAN,

$$\mathrm{ALP}_{\mathrm{end}}(g) = \left|C(g, \phi_{N-1}) - C(g, \phi_0)\right|.$$

C'est le moins coûteux à calculer, mais il peut être bruité puisque toute l'estimation repose sur seulement deux points de mesure, sans robustesse apportée par les $N - 2$ snapshots intermédiaires.

Pour rendre plus robuste cette mesure, on peut moyenner les différences de compétence sur tout l'historique observé :

$$\mathrm{ALP}_{\mathrm{to_now}}(g) = \frac{1}{N-1} \sum_{i=0}^{N-2} \left|C(g, \phi_{N-1}) - C(g, \phi_i)\right|.$$

Cela teste l'hypothèse que la compétence actuelle sur g a dérivé par rapport à l'ensemble de son histoire récente, pas seulement par rapport au point le plus ancien.

Une autre méthode inspirée de (Zhang et al., 2024) est de calculer deux moyennes mobiles exponentielles (EMA) de la trajectoire de compétence avec la même constante de lissage α appliquée deux fois :

$$\mathrm{EMA}_\alpha[i] = \alpha\, C(g, \phi_i) + (1-\alpha)\, \mathrm{EMA}_\alpha[i-1], \qquad \mathrm{recent} = \mathrm{EMA}_\alpha(C), \qquad \mathrm{gradual} = \mathrm{EMA}_\alpha(\mathrm{recent}).$$

recent réagit vite à la trajectoire de C, chaque nouveau point pesant α et les précédents étant décotés géométriquement en $(1-\alpha)^k$ tandis que *gradual*, une EMA appliquée à *recent* elle-même, réagit plus lentement, restant structurellement en retard sur *recent*. Tant que la compétence reste stable, les deux convergent vers la même valeur et leur écart tend vers 0, mais dès que la compétence se met réellement à évoluer, *recent* suit les nouvelles valeurs avant *gradual*, ce qui crée un écart transitoire qui sert de signal de progrès.

La différence entre les deux dernières valeurs de ces deux moyennes, repondérée par une fonction non linéaire f_θ, donne le score :

$$\mathrm{ALP}_{\mathrm{omni}}(g) = \left|f_\theta\big(\mathrm{recent}[N-1]\big) - f_\theta\big(\mathrm{gradual}[N-1]\big)\right|, \qquad f_\theta(p) = \frac{(1-\theta)\, p}{p + \theta(1-2p)}.$$

f_θ est une fonction croissante de $[0, 1]$ dans $[0, 1]$, avec $f_\theta(0) = 0$ et $f_\theta(1) = 1$, (Figure 3A). La valeur de θ par défaut est 0.1, et $f_{0.1}$ dilate l'échelle près de 0 et la comprime près de 1. Ainsi, un même écart absolu de compétence compte davantage lorsque les deux valeurs comparées sont proches de 0 que lorsqu'elles sont proches de 1. C'est exactement l'écart le plus dur à détecter autrement, puisqu'une différence de quelques points de pourcent près de 0% est numériquement minuscule en valeur absolue tout en pouvant signaler un vrai démarrage de l'apprentissage.

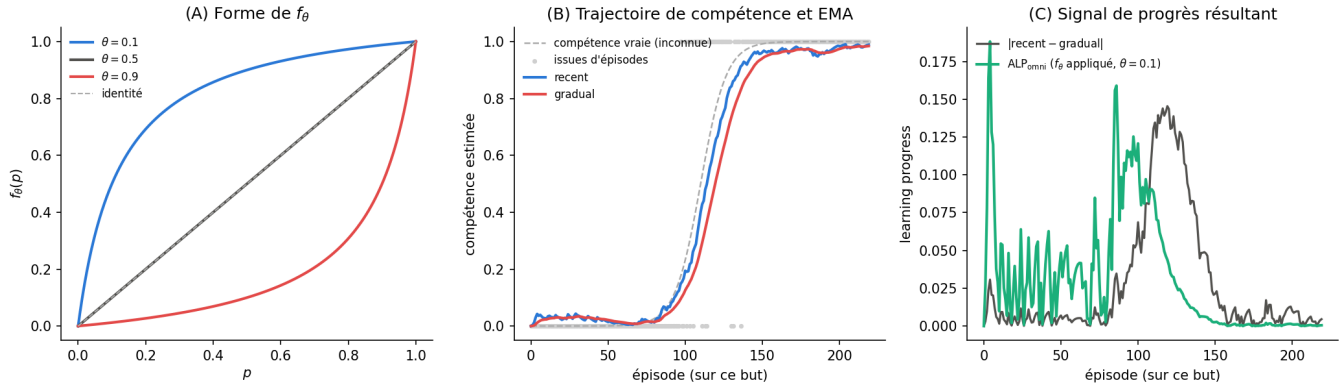


FIGURE 3 – Illustration schématique du mode *omni_ks*. (A) Forme de f_θ pour trois valeurs de θ : concave pour $\theta < 0.5$ (dilate près de 0), identité pour $\theta = 0.5$, convexe pour $\theta > 0.5$ (dilate près de 1). (B) Trajectoire de compétence simulée sur un but dont la vraie probabilité de succès passe progressivement de 0 à 1, avec les deux EMA *recent* (rapide) et *gradual* (lente, en retard). (C) Signal de progrès résultant : le simple écart |recent − gradual| et sa version repondérée ALP_{omni} ($\theta = 0.1$) culminent tous deux pendant la phase de transition et retombent une fois la compétence stabilisée près de 1 ; on observe aussi, tout au début (compétence quasi nulle), une amplification bruitée caractéristique de la dilatation de f_θ près de 0 : un but jamais pratiqué peut y recevoir un score artificiellement élevé par simple bruit d'estimation.

Les trois modes précédents sont *rétrospectifs* car ils ne font que lire des ϕ déjà observés. L'idée de la mesure **prospective_self** développée maintenant est de simuler un pas en avant hypothétique plutôt que de comparer le passé. Pour un but candidat g, on calcule ce que deviendrait ϕ_t si l'agent venait d'essayer g pour chacune des deux issues possibles $r \in \{0, 1\}$. Pour cela, on retire l'entrée la plus ancienne de w_t, en lui substituant hypothétiquement (g, r), et en ré-encodant le résultat $w_t^{(g,r)}$:

$$\phi_t^{(r)} = \rho\big(w_t^{(g,r)}\big), \qquad r \in \{0, 1\}.$$

On combine les deux issues par leur espérance, en les pondérant par la compétence actuellement estimée $p_g := C(g, \phi_t)$:

$$\text{ALP}_{\text{prosp-self}}(g) = p_g \cdot \big|C(g, \phi_t^{(1)}) - C(g, \phi_t)\big| \; + \; (1 - p_g) \cdot \big|C(g, \phi_t^{(0)}) - C(g, \phi_t)\big|.$$

Ce score teste l'hypothèse qu'anticiper le retournement de croyance du réseau sur lui-même est un signal de progrès exploitable, sans attendre de l'observer après coup. Au lieu de faire un pas hypothétique, on pourrait imaginer aussi en faire n, mais cela serait gourmand en calcul, et demande que l'estimateur de compétence

soit déjà très bien calibré.

Une question naturelle après **prospective_self** est de se demander si, au lieu de ne regarder que l'effet d'une tentative sur g sur sa propre compétence estimée, on peut anticiper son effet sur le reste de l'espace des buts. La dernière mesure **prospective_global** tente de répondre à cette question et fonctionne ainsi. Pour un sous-échantillon aléatoire $\mathcal{S}$ de buts, on mesure le déplacement moyen de leur compétence estimée :

$$\mathrm{ALP}_{\text{prosp-glob}}(g) = \Big[\, p_g \cdot \frac{1}{|\mathcal{S}|}\sum_{g' \in \mathcal{S}} \big(C(g', \phi_t^{(1)}) - C(g', \phi_t)\big) \;+\; (1 - p_g) \cdot \frac{1}{|\mathcal{S}|}\sum_{g' \in \mathcal{S}} \big(C(g', \phi_t^{(0)}) - C(g', \phi_t)\big) \,\Big]_+,$$

où $[\cdot]_+ = \max(\cdot, 0)$.

Quel que soit le mode choisi, il reste à convertir le score $\mathrm{ALP}_g(t)$ obtenu pour chaque but en une distribution de probabilité sur $\mathcal{G}$. On utilise une normalisation proportionnelle directe, $p(g) \propto \mathrm{ALP}_g(t)$ où les scores sont simplement divisés par leur somme. Comme pour CURIOUS (Section 2.4), cette distribution est ensuite mélangée à un échantillonnage uniforme : $p_{\text{final}}(g) = (1 - \epsilon)\, p(g) + \epsilon/|\mathcal{G}|$, où ϵ décroît exponentiellement de 1 vers une valeur plancher $\epsilon_{\min}$ au fil de l'entraînement, pour continuer à explorer occasionnellement des buts peu pratiqués dont l'ALP serait sinon mal estimée.

Maintenant que l'estimateur de compétence est développé, ainsi que le mécanisme d'échantillonage de buts, il reste à voir comment entraîner le tout. La sous-section suivante traite de comment ce réseau est entraîné, et comment cet entraînement s'articule avec la boucle principale de PPO.

3.4 Entraînement de l'estimateur de compétence

Le réseau qui implémente $C(g, \phi)$ (c'est-à-dire ψ, τ, les paramètres de l'étape d'agrégation, et la tête finale) est utilisé par deux boucles distinctes. ϕ_t est recalculé à chaque épisode à partir de la fenêtre courante w_t, et sert immédiatement au calcul de l'ALP (Section 3.3) pour choisir le prochain but à pratiquer. Le réseau, en revanche, n'est pas entraîné à chaque épisode, mais périodiquement, tous les κ épisodes, par descente de gradient sur un buffer d'exemples accumulés au fil de l'entraînement $\mathcal{M}$.

Chaque fois que l'agent termine un épisode sur un but g avec un résultat $r \in \{0, 1\}$, le triplet (g, w, r) est ajouté au buffer $\mathcal{M}$ de taille fixe M, implémenté par un deque pour que les plus anciens exemples soient évincés au fur et à mesure. Ici, w est la fenêtre w_t telle qu'elle était juste avant cet épisode, c'est-à-dire l'état de connaissance dans lequel se trouvait l'agent au moment de tenter g. $\mathcal{M}$ grossit donc d'un exemple par épisode, indépendamment du rythme de ré-entraînement du réseau. Ce buffer diffère du buffer de snapshots $\mathcal{N}$ (Section 3.3) de deux façons. D'abord, $\mathcal{M}$ stocke des triplets bruts (g, w, r) utilisés comme exemples pour la perte de compétence, alors que $\mathcal{N}$ stocke des snapshots déjà calculés de ϕ_t lui-même, utilisés uniquement pour mesurer sa dérive dans le temps. Ensuite, $\mathcal{M}$ grossit à chaque épisode, tandis que $\mathcal{N}$ n'est mis à jour que tous les κ épisodes, au même rythme que le réentraînement du réseau.

Chez MAGELLAN, $C_{\theta_t}(g)$ n'est fonction que de g, et sa cible réelle $\mathbb{P}_{\pi_t}(g)$ dépend implicitement de t via π_t. Un exemple (g, r) observé il y a longtemps reste valide pour $\mathbb{P}_{\pi_{t_{\text{ancien}}}}(g)$. Par contre, puisque la politique progresse, $\mathbb{P}_{\pi_t}(g)$ a changé, et ré-entraîner $C_\theta(g)$ sur cet exemple biaise l'estimation vers une

cible désormais obsolète. Il y a donc, chez MAGELLAN, une trade-off entre garder un grand buffer (plus de données, plus stable) et garder un buffer récent (moins de biais de péremption). Avec notre méthode, conditionner sur ϕ permet de lever cette tension. Si ϕ capture réellement tout ce qui compte pour prédire le succès sur g, alors un exemple (g, w, r) reste une supervision valide pour $C(g, \rho(w))$ quel que soit son âge. Un exemple vieux de nombreux épisodes n'est donc pas intrinsèquement plus faux qu'un exemple récent, contrairement à MAGELLAN. Ainsi, le buffer $\mathcal{M}$ peut en principe être indéfiniment grand.

À chaque ré-entraînement, on tire un mini-batch $\mathcal{B} \subset \mathcal{M}$ de taille fixe B et on répète k pas de descente de gradient sur l'entropie croisée binaire :

$$\mathcal{L} = -\frac{1}{|\mathcal{B}|} \sum_{(g,w,r)\in\mathcal{B}} \Big[r \log C\big(g, \rho(w)\big) + (1 - r) \log\big(1 - C(g, \rho(w))\big) \Big].$$

C'est exactement le même principe que l'entraînement du MLP de MAGELLAN, à ceci près que l'entrée du réseau inclut maintenant $\rho(w)$ en plus de g et que w doit donc être ré-encodée à chaque pas de gradient.

Ce fonctionnement épisode par épisode est entièrement asynchrone par rapport à la boucle PPO elle-même. Pendant la collecte d'un rollout, les n environnements tournent en parallèle et terminent leurs épisodes à des instants différents : dès qu'un épisode se termine sur l'un d'eux, le résultat est immédiatement enregistré, et le but suivant pour cet environnement est échantillonné directement, à partir de l'estimateur dans son état le plus à jour. La mise à jour des poids de la politique et du critique de PPO (Section 2.2) n'intervient en revanche qu'une fois à la fin du rollout complet, une fois toutes les transitions des n environnements collectées. Les deux boucles d'apprentissage, curriculum au rythme de l'épisode, PPO au rythme du rollout, ne sont donc jamais explicitement synchronisées l'une à l'autre.

Pour résumer cette architecture et son entraînement, un récapitulatif de tous les paramètres introduits dans cette section, avec leur rôle et leur valeur par défaut, se trouve en Annexe A.

3.5 Discussion : problèmes rencontrés et solutions testées

Le développement du mécanisme présenté ci-dessus a fait émerger des problèmes concrets, chacun avec plusieurs solutions testées, ainsi que des pistes supplémentaires explorées sans être retenues dans la configuration finale.

Premièrement, un exemple du buffer $\mathcal{M}$ associe un but g à une fenêtre w qui, le plus souvent, contient déjà des occurrences récentes de g lui-même. Alors, au lieu d'apprendre à généraliser la compétence sur g à partir de ce qui se passe sur d'autres buts, il peut simplement repérer la présence de g dans w et en lire directement le résultat récent. C'est un raccourci qui minimise très bien la perte d'entraînement, mais qui ne généralise à rien, et ruine l'intérêt même de conditionner sur ϕ : si $C(g, \phi)$ n'apprend qu'à relire g dans ϕ plutôt qu'à inférer sa compétence à partir des autres buts pratiqués, il ne peut plus rien prédire d'utile pour un but absent de la fenêtre.

Pour empêcher ce raccourci, j'ai développé plusieurs idées. D'abord, on peut faire un simple **strip**. Pour un but g précis, ses occurrences sont retirées avant l'encodage dans la fenêtre w vue par la perte principale

$\mathcal{L}$ (jamais à ϕ_t utilisé pour le sampling, qui doit rester le reflet fidèle et complet de l'état de connaissance réel de l'agent). Un simple **strip** est possible quand ρ est invariant par permutation, mais pour la méthode LSTM, cela peut poser problème puisque retirer une occurrence change aussi la position relative de toutes celles qui suivent. C'est pourquoi j'ai aussi développé **mask_embed**, inspiré du token `[MASK]` appris de BERT (Devlin et al., 2019). Au lieu de retirer les occurrences de g, la fenêtre est conservée intacte, mais la sortie $\psi(g_i, r_i)$ de chaque occurrence de g est remplacée par un vecteur appris, partagé entre tous les buts et indépendant du contenu réel de l'occurrence masquée.

Le second problème est le déséquilibre du buffer $\mathcal{M}$. En effet, $\mathcal{M}$ est naturellement très déséquilibré, à la fois entre succès et échecs et entre buts puisqu'un but pratiqué cinq fois plus souvent par le curriculum fournit mécaniquement cinq fois plus d'exemples au buffer. Un tirage uniforme sur $\mathcal{M}$ laisse le gradient du réseau hériter le déséquilibre déjà induit par le choix du curriculum lui-même, c'est-à-dire que les buts déjà beaucoup pratiqués dominent le signal de gradient, au détriment des buts rares, précisément ceux pour lesquels une bonne généralisation serait la plus utile. La solution testée est le **balanced training**. Il pondère le tirage du mini-batch par but plutôt que de tirer uniformément dans $\mathcal{M}$: chaque exemple associé à un but g reçoit un poids $\text{count}(g)^{\alpha-1}$, où $\text{count}(g)$ est le nombre d'exemples associés à g dans $\mathcal{M}$, si bien que la probabilité totale de tirage d'un but donné, sur l'ensemble du buffer, varie en $\text{count}(g)^{\alpha}$. Pour $\alpha = 0$, chaque but a une probabilité équilibrée d'être tirée, pour $\alpha = 1$ on retrouve le tirage uniforme dans $\mathcal{M}$ sans prendre en compte le déséquilibre. Ceci teste l'hypothèse que donner un poids égal à chaque *but* dans le gradient, plutôt qu'à chaque *exemple*, produit un estimateur de compétence qui généralise mieux aux buts rarement visités par le curriculum.

Deux mécanismes additionnels ont par ailleurs été implémentés et testés ponctuellement, sans résultat suffisamment net pour être retenus dans la configuration finale. Pour que ϕ soit interprétable, j'ai ajouté une perte auxiliaire qui décode ϕ en taux de succès par but et une perte contrastive qui rapproche deux sous-échantillons d'une même fenêtre tout en éloignant les fenêtres d'exemples différents du même batch. Les détails de ces deux mécanismes sont donnés en Annexe B.

Le mécanisme de curriculum conditionné sur ϕ étant maintenant complètement défini, il faut maintenant le confronter à des problèmes concrets. C'est l'objet de la section suivante, qui présente les trois environnements développés pendant ce stage pour le tester.

4 Environnements

Cette section décrit les trois différents environnements développés lors du stage. Pour chacun, le design a évolué tout au long du stage, et on ne présente ici que les versions finales, dans l'ordre chronologique de développement.

4.1 MiniGrid

Le premier environnement développé se base sur MiniGrid (Chevalier-Boisvert et al., 2023), une librairie Python très utilisée en Reinforcement Learning. Elle permet de créer des grilles avec différents objets comme des murs, portes, clés, coffres, ou encore balles, de différentes couleurs. J'ai donc utilisé cette librairie pour construire mon propre environnement, une sorte de labyrinthe rectangulaire composé de plusieurs salles. Chaque salle, composée de cases, est elle-même rectangulaire, et permet d'accéder aux salles juste au-dessus et juste à droite grâce à des clés et des portes de couleurs. Il y a 6 couleurs possibles, et une porte d'une certaine couleur ne peut être ouverte qu'avec une clé de cette même couleur. L'agent spawn toujours dans la case en bas à gauche de la salle en bas à gauche, i.e. en $(0, 0)$. On peut donc générer de telles grilles en variant leur taille, qu'on définit par quatre valeurs : le nombre de salles par ligne, le nombre de salles par colonne, le nombre de cases par ligne d'une salle, et le nombre de cases par colonne d'une salle. On note la grille (N, M, n, m). Un exemple pour $(3, 3, 5, 5)$ est présenté sur la Figure 4.

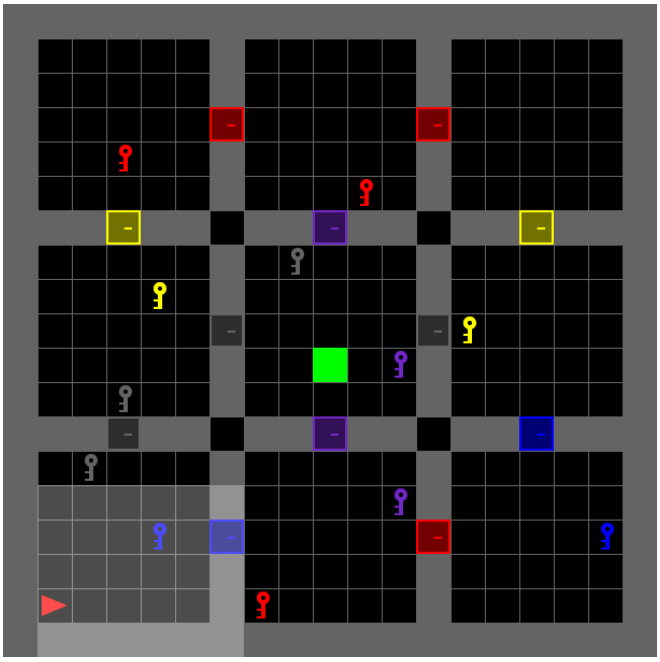


FIGURE 4 – Rendu de MiniGrid pour une grille $(3, 3, 5, 5)$

Les actions disponibles dans cet environnement sont au nombre de 6 : *turn right*, *turn left*, *move forward*, *pick*, *drop* et *toggle*. J'ai défini $\mathcal{G}$ comme l'ensemble des cases accessibles, et chaque but est implémenté par ses coordonnées (x, y) correspondant à la case, qu'on ramène ensuite sur $[0, 1] \times [0, 1]$. Le nombre de goals est donc $N \times M \times n \times m - 1$ (en enlevant la case de spawn). Sur l'exemple de la Figure 4, l'agent

doit atteindre la case verte, et donc doit, par exemple, naviguer jusqu'à la clé bleue, *pick* pour la prendre, atteindre la porte bleue, *toggle* pour l'ouvrir, *drop* la clé bleue un peu plus loin pour pouvoir prendre la clé violette (l'agent ne peut porter qu'une seule clé à la fois), puis de la même manière ouvrir la porte violette et atteindre la case verte, i.e. le goal.

J'ai défini la longueur maximale d'un épisode par $T = 2(n + m)(N + M)$. Le terme $(n + m)$ donne la marge nécessaire pour traverser une salle dans n'importe quelle direction, et $(N + M)$ celle pour traverser la grille de salles jusqu'au but le plus éloigné. Le facteur 2 laisse à l'agent une marge au-delà du plus court chemin théorique pour le contournement d'obstacles, la récupération de clés situées à l'écart du trajet direct, etc. Un épisode se termine donc de deux façons : soit l'agent atteint la case cible en moins de T steps, et l'épisode se termine alors immédiatement par un succès, avec une récompense valant 1, soit T steps se sont écoulés sans que l'agent atteigne la case objectif, et l'épisode se termine par un échec, avec une récompense valant 0. Cette récompense binaire, non nulle uniquement en cas de succès, est précisément ce qui rend le signal *sparse* au sens de la Section 2.3 : plus l'espace des buts est grand, plus un agent agissant au hasard a peu de chances de recevoir ce signal.

On observe que plus la case goal est dans une salle en haut à droite, plus le goal est difficile à atteindre. C'est d'ailleurs ce gradient de difficulté, couplé au design de l'espace des goals, qui fait l'intérêt de cet environnement : il teste si l'estimateur de compétence généralise correctement en fonction de la distance structurelle entre buts même pour des buts peu ou jamais pratiqués directement.

Pour quantifier précisément cette notion de difficulté, on repère la salle contenant un but g par sa position $(i, j) \in \{0, \dots, N-1\} \times \{0, \dots, M-1\}$ (i vers la droite, j vers le haut). On définit alors sa distance de salle au spawn comme la distance de Manhattan entre salles :

$$d(g) = i + j.$$

Cette distance correspond exactement au nombre minimal de portes que l'agent doit ouvrir pour atteindre g depuis le spawn, puisque chaque salle n'est connectée qu'à sa voisine de droite et sa voisine du dessus. C'est cette métrique, et non une distance euclidienne dans l'espace $[0, 1]^2$ des coordonnées (x, y) normalisées, qui reflète la vraie difficulté structurelle d'un goal : deux cases peuvent être proches en (x, y), par exemple de part et d'autre d'un mur de salle, sans être du tout proches en nombre de portes à franchir.

L'observation de l'agent est composée de deux parties, concaténées avant d'être passées à la tête acteur-critique :

— une vue locale partielle $7 \times 7 \times 3$ (format symbolique standard de MiniGrid : type d'objet, couleur, état par case, pas des pixels RGB), centrée sur l'agent et orientée selon sa direction, donc, selon la taille de la salle, l'agent ne voit pas forcément toute la salle d'un coup ;

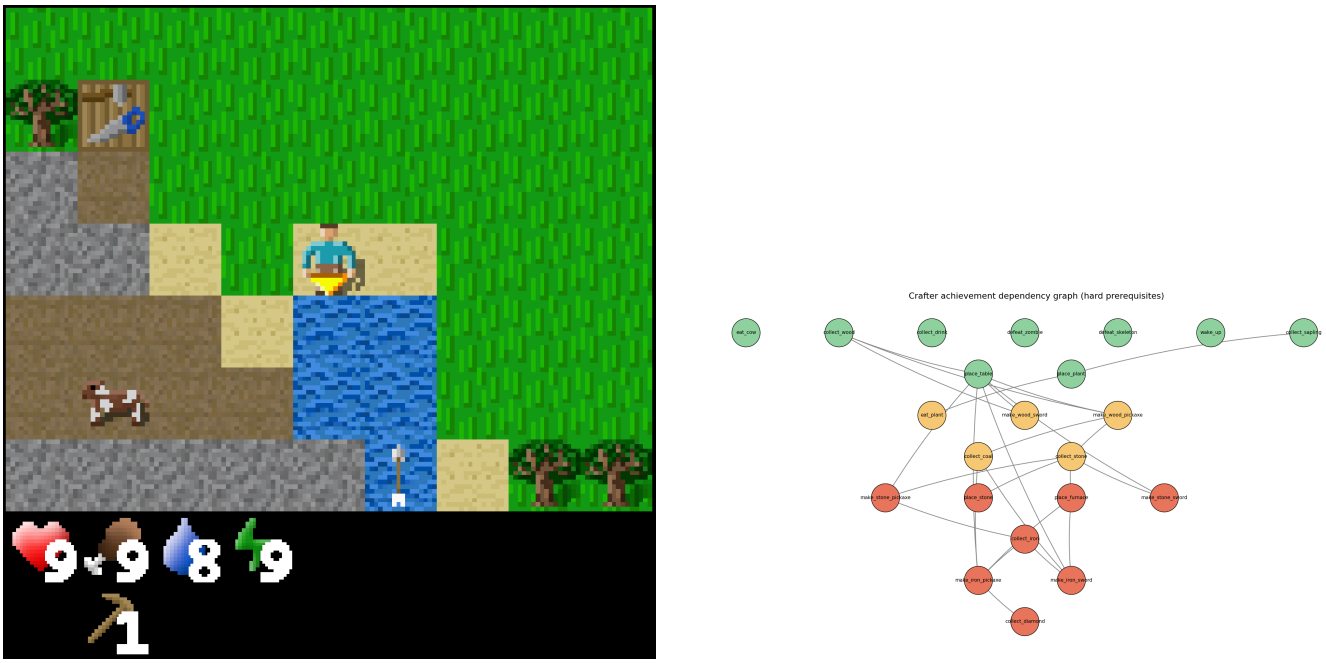
— sa position absolue (x, y) sur la grille globale normalisée et sa direction (dx, dy), ajoutées directement à l'encodage sans passer par un réseau convolutif.

La vue locale est encodée par un petit réseau convolutif (3 couches, $16 \to 32 \to 64$ canaux, noyaux 2×2) suivi de deux couches entièrement connectées ; sa sortie est ensuite concaténée au vecteur position/direction avant d'être combinée au but (x, y) cible dans la tête acteur-critique. Le fait que l'agent connaisse toujours sa

position absolue, et pas seulement ce qu'il voit localement, explique en grande partie pourquoi une politique sans mémoire (sans LSTM) suffit dans nos expériences : l'agent n'a pas besoin de mémoriser où il se trouve, seulement d'apprendre à utiliser cette position et ce qu'il voit localement pour choisir la bonne action vers un but donné.

4.2 Crafter

Le deuxième environnement est Crafter (Hafner, 2022), un jeu où l'agent évolue dans un monde 2D procédural ressemblant à Minecraft, comme le montre la Figure 5. L'agent doit survivre en subvenant à ses besoins vitaux, boire, manger et dormir, modélisés par des jauges qui diminuent avec le temps. Pour remonter ces jauges, l'agent peut respectivement boire de l'eau, manger une vache ou une plante, et dormir. Le monde contient également des créatures hostiles, zombies et squelettes, qui attaquent le joueur lorsqu'il s'approche d'elles. Les zombies attaquent au corps à corps et les squelettes à distance. Le joueur dispose de 9 points de vie initiaux, qu'il perd à chaque attaque subie, et si sa vie atteint 0, il meurt. Pour se défendre, il peut utiliser soit ses mains, ce qui inflige peu de dégâts, soit une épée (en bois, en pierre ou en fer, la puissance augmentant avec le matériau).



(a) Rendu de l'environnement

(b) Arbre de dépendance des buts

FIGURE 5 – Crafter : rendu de l'environnement et arbre de dépendance des 22 buts.

L'agent dispose de 17 actions : ne rien faire, se déplacer dans les 4 directions, une action contextuelle *do* qui ramasse, attaque ou récolte selon ce qui fait face à l'agent, dormir, poser une pierre, une table d'artisanat, un four ou une plante, et fabriquer une pioche ou une épée dans l'un des trois matériaux (bois, pierre, fer).

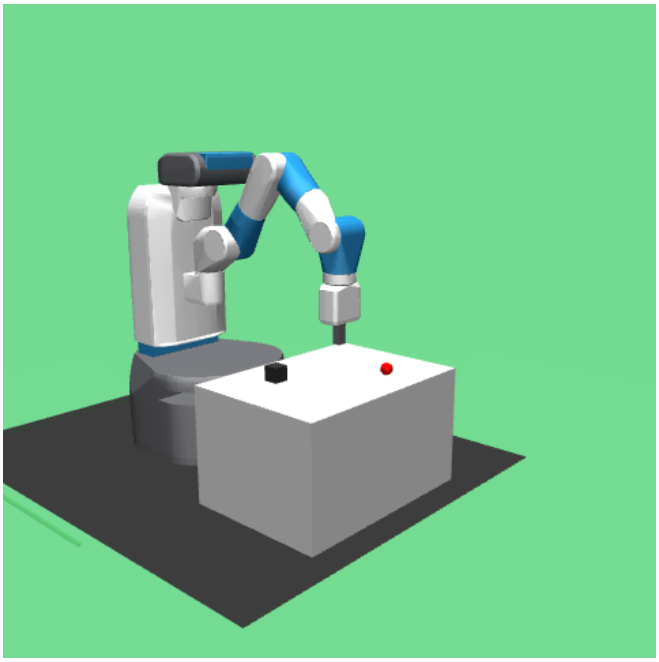
Dans mon implémentation, l'ensemble des buts est l'ensemble des *achievements* possibles du jeu, au nombre de 22 (Figure 5, à droite). L'intérêt de Crafter est qu'il permet d'avoir un espace de buts structuré par

un véritable arbre de dépendance. Par exemple, fabriquer une pioche en fer suppose d'avoir déjà collecté du bois, de la pierre et du charbon, puis fabriqué une pioche en pierre. Sur les 22 buts, 14 ne dépendent que de la fabrication et peuvent être isolés des 8 buts liés à la survie et au combat (boire, manger, dormir, planter, ou vaincre un zombie/squelette). Désactiver ces derniers teste le curriculum sur l'arbre de dépendance seul : la survie ajoute une seconde source de difficulté qui n'a rien à voir avec la structure de l'espace des buts elle-même, et pourrait donc masquer l'effet propre du conditionnement sur ϕ si les deux restaient mélangés. Dans les deux modes (avec ou sans survie), lorsque le but sur lequel l'agent s'entraîne pendant l'épisode est atteint, l'agent reçoit une récompense valant 1, et s'il meurt, ou si le nombre maximal de pas (fixé à 10000 avec survie et 300 sans survie) est atteint avant que le but ne soit accompli, la récompense reçue est 0. On retrouve à nouveau une récompense binaire et rare, pour les mêmes raisons qu'en Section 2.3.

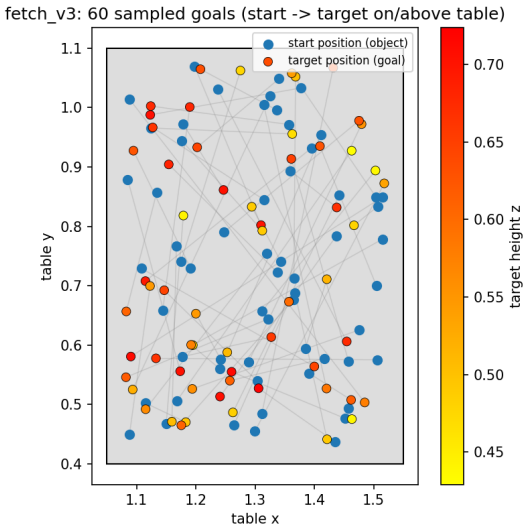
Pour représenter ces buts, plusieurs solutions sont envisageables. La plus simple est un *one-hot encoding*, mais celui-ci ne permet aucune représentation des relations entre buts, ce qui prive l'estimateur de compétence de tout bénéfice tirable de la structure de l'espace des buts décrite ci-dessus. Une alternative, à la manière de Gaven et al. (2025), consiste à utiliser un LLM pour encoder la description textuelle de chaque but, ce qui devrait produire une représentation plus riche, mettant en valeur les liens entre buts proches. J'ai essayé cette idée en encodant uniquement le nom du but, mais l'entraînement s'en trouvait ralenti sans meilleur résultat, j'ai donc retenu une troisième approche, une représentation construite directement à partir de l'arbre de dépendance lui-même (détails en Annexe C).

L'encodeur visuel choisi est un NatureCNN suit celui de (Mnih et al., 2015) (3 couches convolutives, 8×8/stride $4 \rightarrow 4 \times 4$/stride $2 \rightarrow 3 \times 3$/stride 1, suivies d'une couche entièrement connectée) qui projette l'observation dans un espace latent de dimension 512. L'agent voit une fenêtre locale de 9×7 cases centrée sur lui, complétée en dessous par deux rangées supplémentaires affichant sa vie, ses jauges d'eau, de nourriture et de sommeil, ainsi que son inventaire, soit 9×9 cases au total, rendues sous forme d'une image RGB 64×64 (chaque case fait donc environ $64/9 \approx 7$ pixels de côté). L'approche LSTM a aussi été envisagée, mais outre le coût de mémoire supplémentaire, l'entraînement du CNN est déjà nettement plus lent que celui des encodeurs, plus simples, des deux autres environnements.

4.3 Fetch



(a) Rendu de l'environnement



(b) Schéma de l'espace des buts

FIGURE 6 – Fetch : rendu de l'environnement et schéma de l'espace des buts.

Le troisième et dernier environnement développé dans ce stage utilise Gymnasium-Robotics (de Lazcano et al., 2024), une librairie très utilisée en Reinforcement Learning pour la robotique, reposant sur le moteur physique MuJoCo. Elle propose notamment deux familles de tâches : les environnements *Dexterous Hand*, où l'agent est une main robotique qui manipule un objet, et les environnements *Fetch* (Plappert et al., 2018), qui présentent un bras articulé devant une table, où différentes tâches sont possibles (Figure 6). J'ai choisi de développer l'environnement *Fetch* avec la tâche *pick and place*, la plus difficile des tâches proposées. Le bras articulé doit y attraper une balle posée sur la table et la déposer à un point ciblé.

Contrairement à *MiniGrid* et *Crafter*, cet environnement est à espace d'actions et d'états continus, ce qui rend naturellement l'espace des buts lui-même continu. Un but est défini par le point de départ $O = (x_0, y_0, h)$ de la balle (posée sur la table de hauteur fixe h) et le point cible $C = (x_c, y_c, z_c)$ où elle doit être déposée. z_c peut dépasser h jusqu'à une hauteur maximale $\Delta z_{\max} = 0.30\text{m}$. x_0, y_0, x_c, y_c sont tous les quatre tirés uniformément sur la même zone rectangulaire définit par la table. h étant fixe, un but est donc entièrement déterminé par le quintuplet libre $(x_0, y_0, x_c, y_c, z_c) \in \mathbb{R}^5$. En pratique, l'estimateur de compétence utilise le sextuplet $(x_0, y_0, h, x_c, y_c, z_c)$ par simplicité d'implémentation. Cet espace de buts $\mathcal{G}$ est donc infini, alors que notre algorithme de curriculum ne fonctionne que pour un ensemble fini de buts. Pour s'y ramener, je fixe un nombre de buts souhaité n et j'échantillonne uniformément n buts dans ce pavé au début de l'entraînement (Figure 6). Ceci permet d'obtenir autant de buts que souhaité sans que la difficulté intrinsèque de la tâche ne change avec n, contrairement à *MiniGrid* où agrandir ou multiplier les salles pour augmenter le nombre de buts augmente aussi la difficulté.

Le choix de Fetch est aussi motivé par le fait qu'il s'agisse, cette fois, d'un environnement 3D et non 2D comme les deux précédents, et que l'agent ne soit plus un personnage qui se déplace dans l'espace mais un bras robotique amenant un objet d'un point O à un point C. D'ailleurs, il suffit que le centre de la balle soit à une distance inférieure à $\epsilon = 0.05$m du point C pour que le but soit validé et que l'agent reçoive une récompense sparse de 1. S'il n'y parvient pas en $T = 200$ pas, il reçoit une récompense nulle.

L'action native de l'environnement est un vecteur continu à 4 dimensions $(dx, dy, dz, g) \in [-1, 1]^4$ dont les trois premières composantes indiquent le déplacement cartésien du bras, et où la quatrième correspond à l'ouverture de la pince. On pourrait en théorie utiliser un algorithme de Reinforcement Learning adapté à cet espace d'action continu, mais ce n'est pas l'objet de cette étude, qui utilise partout une politique catégorielle. Par simplicité, et pour rester cohérent avec les deux premiers environnements, on discrétise donc l'espace d'actions. dx, dy, dz sont chacun ramenés à 3 niveaux $\{-0.5, 0, 0.5\}$, et la pince à 2 états, ouverte ou fermée, ce qui donne $3 \times 3 \times 3 \times 2 = 54$ actions discrètes. Les niveaux de déplacement ± 0.5 correspondent à un déplacement réel d'environ 0.025m dans un sens ou dans l'autre, une valeur calibrée pour que la pince puisse bien attraper l'objet et le soulever.

Ce qui est pratique avec cet environnement, c'est qu'il ne nécessite pas de CNN pour encoder l'observation, un simple MLP suffit. En effet, l'observation native de Gymnasium-Robotics est un vecteur de 25 dimensions. Il indique les positions et vitesses de la pince et de la balle, l'état d'ouverture de la pince, et la rotation de la balle. Ainsi, l'état de la scène est entièrement décrit sans qu'une image ne soit nécessaire. Cette observation de 28 dimensions est donc directement encodée par un MLP à 3 couches, avant d'être combinée au but $(x_0, y_0, h, x_c, y_c, z_c)$ cible dans la tête acteur-critique, de la même manière que pour les deux environnements précédents.

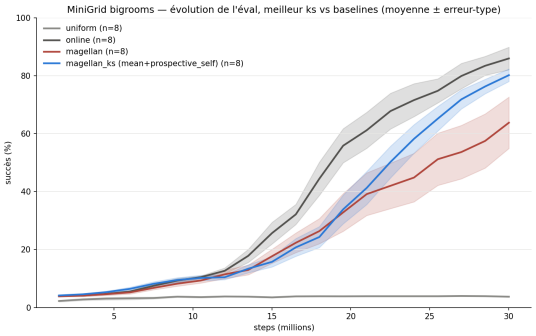
Ces trois environnements couvrent ainsi des régimes volontairement très différents : un espace de buts discret et structuré géométriquement (MiniGrid), un espace discret structuré par un arbre de dépendance (Crafter), et un espace continu de grande dimension, de taille librement réglable (Fetch). La section suivante présente les résultats obtenus sur chacun d'eux.

5 Résultats

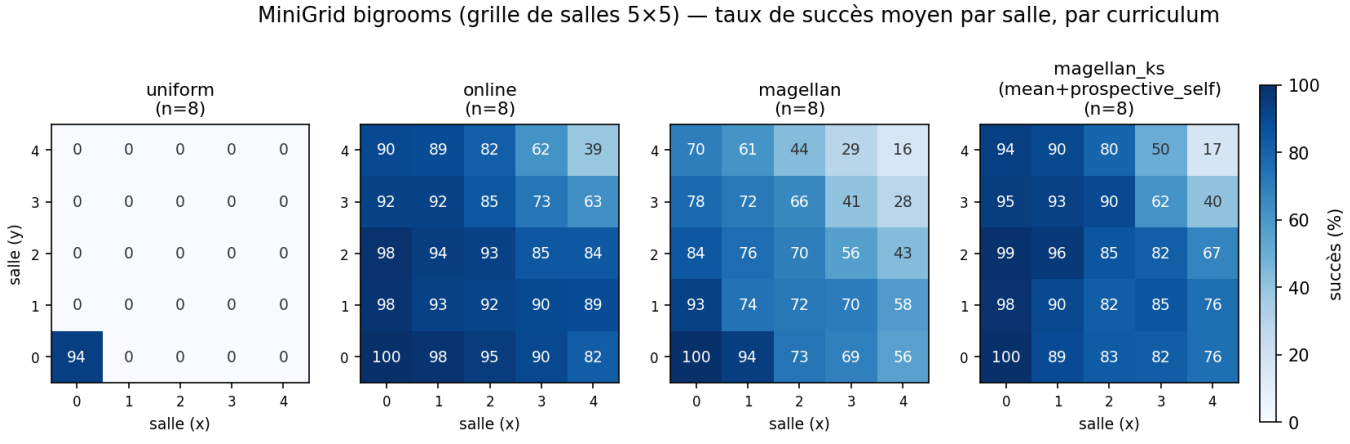
MiniGrid est l'environnement sur lequel le plus de tests ont été menés car son encodeur est un simple MLP contrairement au CNN de Crafter, ce qui le rend nettement plus rapide à entraîner, et sa structure spatiale offre un vrai gradient de difficulté entre buts contrairement à Fetch, où tous les buts sont a priori comparables en difficulté. *magellan_ks* désigne, dans toutes les figures et tableaux qui suivent, l'approche développée dans ce stage, MAGELLAN conditionné sur l'état de connaissance ϕ (Section 3). Dans chacun des trois environnements, on compare magellan_ks à *uniform* pour mesurer l'intérêt d'un curriculum, *online* (ALP de CURIOUS) et *magellan*.

5.1 MiniGrid

La configuration principale étudiée est une grille $(5, 5, 8, 8)$ (notation introduite en Section 4). Pour cette taille, $|\mathcal{G}| = 1599$ et $T = 320$, et on entraîne pour 30M pas, ce qui donne au minimum 93 750 épisodes, soit 59 épisodes par but en moyenne, ce qui est suffisant pour qu'online reste compétitif, comme on le voit ci-dessous.



(a) Taux de succès moyen (bande = erreur-type), évalué à 20 checkpoints réguliers au cours de l'entraînement.



(b) Taux de succès moyen par salle au checkpoint final.

FIGURE 7 – MiniGrid $(5, 5, 8, 8)$, *uniform*, *online*, *magellan* et *magellan_ks* (pool_mode=mean, ALP=prospective_self).

magellan_ks termine l'entraînement à 80.2% de succès en moyenne contre 63.8% pour magellan, 86.0% pour online et 3.7% pour uniform. Pour évaluer la fiabilité de l'écart magellan_ks/magellan malgré le faible nombre de seeds et une distribution bimodale (certaines seeds de magellan s'effondrant près de 0%), on utilise un test de rang Mann-Whitney, H_1 : magellan_ks > magellan, indépendamment à chacun des 20 checkpoints, suivant la pratique de Zhang et al. (2024).

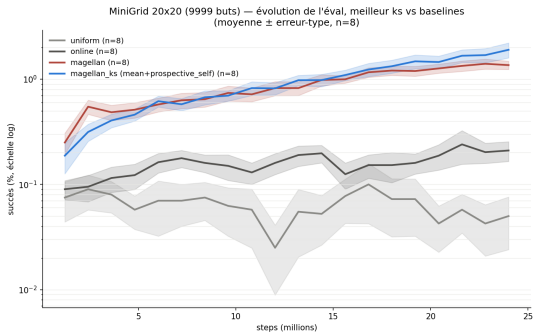
Le résultat est majoritairement favorable à magellan_ks sans franchir le seuil de 5% (au checkpoint final, $p = 0.080$). magellan_ks réduit surtout nettement la variance magellan_ks réduit surtout nettement la variance inter-seed comparé à magellan, avec un écart-type de 6.5 points contre 26.8.

L'espace de buts de MiniGrid étant une grille de salles, le taux de succès final peut aussi être décomposé salle par salle plutôt que réduit à une seule moyenne globale, comme le montre la Figure 7b pour les quatre curricula. uniform ne réussit que la salle de spawn et n'atteint jamais aucune autre salle . Donc, sans curriculum, l'agent n'explore jamais assez loin pour apprendre à atteindre les salles distantes. online et magellan_ks couvrent toute la grille avec une décroissance progressive en fonction de la distance au spawn (de 94–100% sur les salles proches à 17–39% sur la salle la plus éloignée), magellan décroissant plus vite que les deux (100% à 16%).

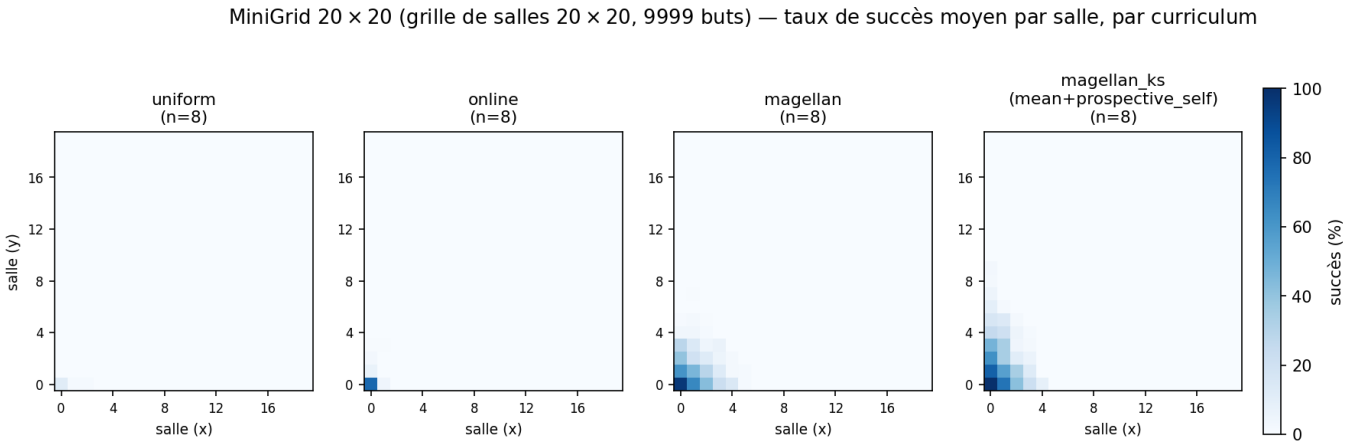
En théorie, l'avantage de magellan et magellan_ks sur online se manifeste surtout quand $|\mathcal{G}|$ est grand. Pour visualiser cet effet, on reprend la même expérience sur une grille de 20×20 salles de 5×5 cases, soit 9999 buts, avec $T = 800$ pas max par épisode et 30M pas d'entraînement, soit seulement 4 épisodes par but en moyenne ce qui est trop peu pour online.

Le niveau absolu de succès reste faible à cette échelle ($\sim$ 1–2%, Figure 8a) ce qui est attendu vu le nombre de buts et le budget de pas limité. L'important est la différence entre curricula, et magellan_ks bat tous les autres. Contre online, le même test de rang checkpoint par checkpoint (H_1 : magellan_ks > online) confirme que l'écart n'a rien d'un artefact puisque $p < 0.004$ dès le deuxième checkpoint. Contre magellan, magellan_ks reste également devant en fin d'entraînement (1.90% contre 1.36%), mais l'écart n'est pas significatif, le nombre de seeds étant trop limitée. La Figure 8b montre qu'online ne réussit que la salle de spawn, que magellan commence à apprendre quelques salles supplémentaires autour du spawn et que magellan_ks va encore un peu plus loin que magellan dans la grille.

Le Tableau 5 (Annexe D) étend la comparaison à l'ensemble des configurations magellan_ks testées à cette échelle. Toutes les architectures de pooling testées battent magellan (1.36%) à 9999 buts, chacune restant dans la fourchette 1.4–1.9%.



(a) Taux de succès moyen (échelle log, bande = erreur-type), évalué à 20 checkpoints réguliers au cours de l'entraînement.



(b) Taux de succès moyen par salle au checkpoint final.

FIGURE 8 – MiniGrid $(20, 20, 5, 5)$, *uniform*, *online*, *magellan* et l*magellan_ks* (pool_mode=mean, ALP=prospective_self).

Plusieurs leviers indépendants de l'architecture de ρ ont été testés. On présente ici la correction de fuite d'auto-occurrence (*strip* / *mask_embed*, Section 3.5), la taille du buffer de réentraînement M, la taille de fenêtre K, et le mode de calcul du learning progress (ALP).

La Figure 9 compare, pour chaque méthode d'agrégation, l'absence de correction de fuite d'auto-occurrence, *strip* et *mask_embed*. L'effet est fortement dépendant de la méthode d'agrégation. *strip* aide nettement mean (46.2% → 76.9%) et transformer (26.8% → 62.4%), reste neutre pour attention (46.5% → 47.8%), et nuit à lstm (60.2% → 32.7%). mean et transformer sont tous deux invariants par permutation et retirer l'item en autoréférence ne change rien à la façon dont les autres items sont traités, et libère la moyenne ou l'attention de la distraction que représentait ce biais, pour se concentrer entièrement sur les autres buts de la fenêtre. lstm, à l'inverse, encode un ordre chronologique et *strip* perturbe directement la structure temporelle que lstm exploite, d'où la dégradation. *mask_embed*, qui préserve la position dans la séquence en ne masquant que le contenu de l'item, est mieux géré par lstm que *strip* (53.8% contre 32.7%), sans pour autant faire mieux que l'absence totale de correction (60.2%) . attention ne change presque pas quelle que soit la correction appliquée (46.5% sans, 47.8% avec strip, 48.1% avec mask_embed). On aurait

pu penser que son mécanisme de pondération apprise α_i (Section 3.2) apprend de lui-même à sous-pondérer l'item en autoréférence, rendant la correction explicite redondante. Une sonde directe sur les poids α_i réfute cette hypothèse. C'est l'inverse qui se produit (Annexe E), sans que cela explique pour autant l'insensibilité d'attention à la correction.

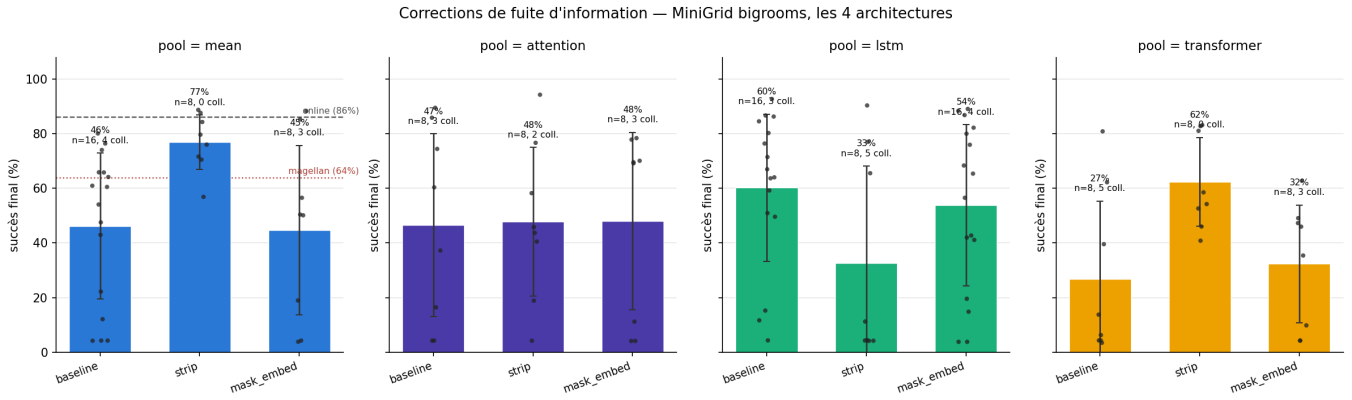


FIGURE 9 – MiniGrid $(5, 5, 8, 8)$, effet de la correction de fuite d'auto-occurrence par méthode d'agrégation, taux de succès final.

La Figure 10 explore l'effet de la taille M du buffer d'exemples $\mathcal{M}$ (Section 3.4), sur une grille plus restreinte $(5, 5, 5, 5)$. Comme attendu par l'argument de stationarité (Section 3.1), conditionner sur ϕ rend en principe un exemple (g, w, r) valide indépendamment de son âge, donc M ne devrait pas avoir d'effet systématique une fois suffisamment grand. C'est globalement ce qu'on observe. Les courbes des trois architectures restent stables sur toute la plage de M testée, contrairement à MAGELLAN où un buffer trop grand biaiserait l'estimation vers des cibles $\mathbb{P}_{\pi_{t_{\text{ancien}}}}(g)$ désormais obsolètes (Section 3.4), ce qui impose d'y choisir un buffer récent plutôt que grand. Cette insensibilité à M est donc elle-même une confirmation indirecte, empirique, que ϕ capture raisonnablement bien ce qui compte pour prédire le succès, au moins assez pour que de vieux exemples restent exploitables.

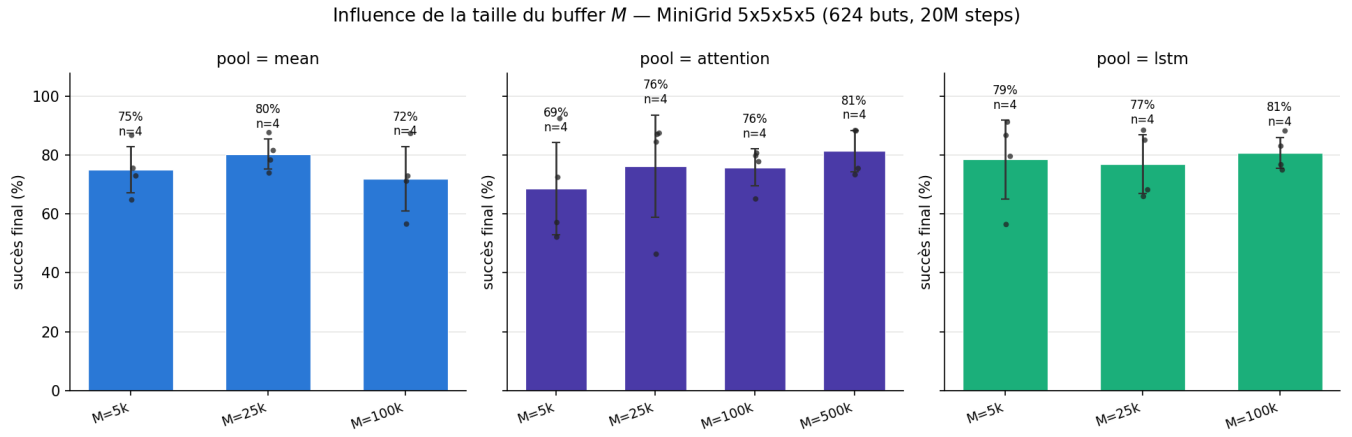


FIGURE 10 – MiniGrid $(5, 5, 5, 5)$, effet de la taille du buffer d'entraînement M par architecture.

Le Tableau 1 résume le balayage sur la taille de fenêtre K (nombre d'items résumés par ϕ, Section 3.1),

sur chaque architecture et sa meilleure configuration propre. Le résultat ne correspond pas à l'intuition de départ. On s'attendait à ce que pour représenter fidèlement l'état de connaissance à l'instant t, il faut s'appuyer sur des items récents, et pas des anciens qui ne correspondraient plus à l'état réel de l'agent. mean n'a aucun moyen de tenir compte de cette ancienneté, alors que lstm et transformer le peuvent en principe. Agrandir K ne devrait donc pas pénaliser ces deux dernières architectures, et pourrait même leur apporter davantage d'information. Ce n'est pas ce qu'on observe. En effet, les trois architectures montrent au contraire un dégradement progressif en augmentant K. Seul mean remonte nettement pour à K=1000 (67.8%). Cette dégradation suggère donc qu'un autre facteur que la seule capacité à filtrer les items anciens est à l'œuvre, possiblement un effet d'optimisation puisque une fenêtre de taille intermédiaire change aussi la difficulté effective de la tâche de régression apprise par ψ et τ.

TABLE 1 – MiniGrid $(5, 5, 8, 8)$, balayage sur la taille de fenêtre K, sur la meilleure configuration connue de chaque architecture

	$K = 30$	$K = 100$	$K = 300$	$K = 1000$
mean	61.5% ± 32.7 (n=8)	49.6% ± 21.9 (n=8)	44.2% ± 36.2 (n=8)	67.8% ± 21.3 (n=8)
lstm	60.2% ± 27.8 (n=16)	53.4% ± 28.3 (n=8)	52.8% ± 34.6 (n=8)	52.7% ± 34.9 (n=8)
transformer	62.4% ± 17.3 (n=8)	37.7% ± 25.6 (n=8)	41.7% ± 29.6 (n=8)	29.9% ± 34.7 (n=8)

Le Tableau 2 compare pour mean les différents modes de calcul de l'ALP présentés en Section 3.3, dans leur ordre de présentation. prospective_self domine très largement les autres modes. prospective_global s'effondre au niveau du plancher quasi-uniforme (4.0%) parce que l'effet d'un seul essai hypothétique sur la compétence moyenne de tous les buts est un signal beaucoup trop bruité pour être exploitable. L'écart entre preospective_self et les autres modes ne vient pas tant d'une moyenne plus élevée que d'une absence quasi totale d'effondrement. Là où les autres modes affichent une dynamique bimodale, prospective_self semble être le seul signal d'ALP suffisamment stable pour éviter systématiquement ce mode d'échec, comme le montrent les écarts-types.

TABLE 2 – MiniGrid $(5, 5, 8, 8)$, pool_mode=mean, balanced_training activé, comparaison des modes ALP, taux de succès final (moyenne ± écart-type).

Mode ALP	Succès final
endpoints	46.2% ± 27.6 (n=16)
to_now	41.9% ± 34.2 (n=7)
omni_ks ($\theta = 0.1$)	37.5% ± 13.0 (n=8)
omni_ks ($\theta = 0.5$)	32.5% ± 38.1 (n=8)
prospective_self	**80.2**% ± 6.5 (n=8)
prospective_global	4.0% ± 0.3 (n=8)

5.2 Que représente l'état de connaissance ϕ ?

Au-delà de la performance du curriculum, une question distincte se pose : ϕ encode-t-il effectivement une information utile et structurée sur l'état de compétence de l'agent ? Cette sous-section présente trois sondes construites spécifiquement pour répondre à cette question sur MiniGrid, dont la structure spatiale connue permet de définir une notion de référence contre laquelle comparer ϕ.

Pour isoler ce que ϕ encode à propos d'une salle précise, on synthétise, pour chaque salle s, une fenêtre w de $K = 30$ items identiques $(g_s, 1)$, c'est-à-dire que tous les essais récents de l'agent portent sur s, et ont tous réussi. On encode ensuite $\phi_s = \rho(w)$ pour les 25 salles de la grille, et on projette ces 25 vecteurs par ACP en 2 dimensions. On répète cette opération à plusieurs checkpoints du même entraînement.

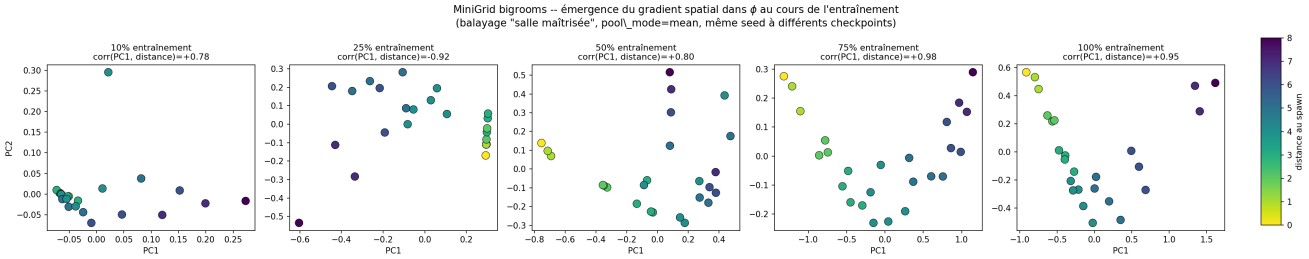


FIGURE 11 – MiniGrid $(5, 5, 8, 8)$, projection ACP des ϕ_s synthétiques, à 5 moments de l'entraînement, colorée par distance de salle au spawn.

La corrélation entre la première composante principale (PC1) et la distance de salle au spawn augmente progressivement au cours de l'entraînement (+0.78 à 10%, jusqu'à +0.95–0.98 à partir de 75% de l'entraînement). ϕ n'organise donc pas cette structure spatiale dès l'initialisation, elle se construit progressivement avec l'apprentissage. Au fil de l'entraînement, les salles apprises s'alignent aussi de plus en plus nettement selon un arc. Les salles proches du spawn se regroupent à une extrémité, les plus éloignées à l'autre, et les salles à distance intermédiaire se répartissent entre les deux plutôt que de rester dispersées au hasard. La distance de salle au spawn devient donc progressivement le facteur dominant de l'organisation de ϕ, pas seulement un facteur parmi d'autres.

La sonde précédente ne regarde que la compétence prédite sur la salle maîtrisée elle-même. On teste maintenant si la maîtrise apparente d'une salle A se transfère à une autre salle B. A partir d'une fenêtre synthétique A maîtrisée, on encode ϕ_A puis on interroge le réseau de compétence sur chaque salle B de la grille, $\hat{C}(g_B, \phi_A)$. La compétence prédite sur B reste élevée pour de petites distances (A, B) et décroît progressivement à mesure que (A, B) s'éloignent, sans effondrement brutal au-delà de la diagonale. ϕ généralise donc continûment dans l'espace des salles, et pas seulement de façon binaire sur la salle littéralement pratiquée.

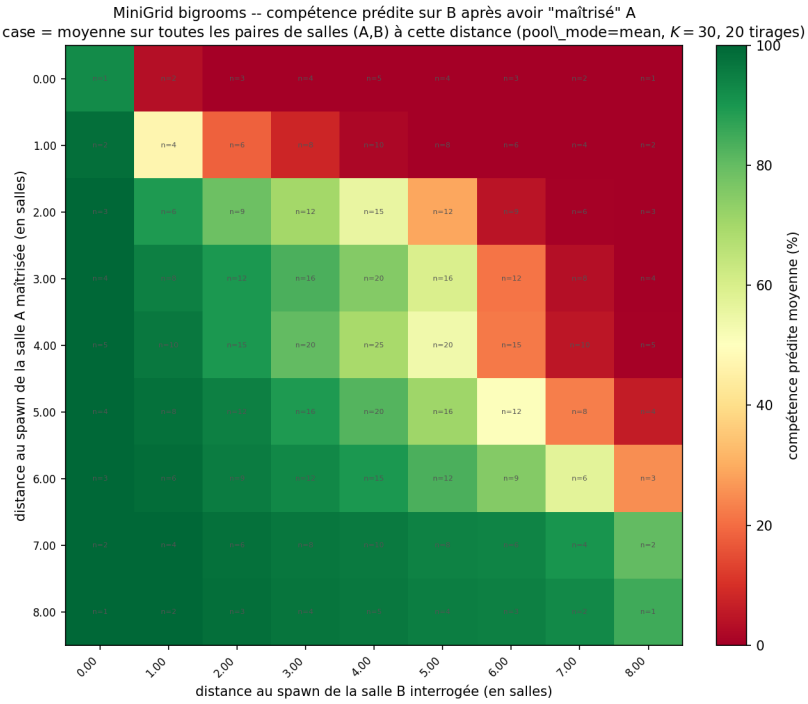


FIGURE 12 – MiniGrid $(5, 5, 8, 8)$, compétence prédite sur B après avoir maîtrisé A, moyennée sur toutes les paires de salles (A, B) partageant la même distance de salle au spawn.

Les deux sondes précédentes utilisent des fenêtres synthétiques entièrement homogènes. En pratique, une fenêtre réelle w_t mélange des buts et des résultats divers. Pour tester si ϕ reste un résumé stable malgré cette composition variable, on définit un front de déblocage F, représentant la distance de salle au-delà de laquelle le succès devient improbable, avec une transition douce (probabilité de succès $p(d, F) = \text{clip}((F - d)/1 + 0.5, 0, 1)$ pour une salle à distance d). Pour chaque F, on tire 40 fenêtres indépendantes de $K = 30$ salles aléatoires, chacune étiquetée succès/échec selon $p(d, F)$, et on encode chacune séparément pour conserver la dispersion liée au tirage aléatoire des salles de la fenêtre. On montre le résultat pour une seed réussie et une seed qui s'est effondré en Figure 13. Pour les deux seeds, les deux premières composantes principales résument à elles seules la quasi-totalité de la variance de ϕ (95.7% pour le seed réussi, 94.8% pour le seed effondré), donc la projection 2D n'occulte pas une structure cachée en dimension supérieure. Cependant, la façon dont les points s'y organisent diffère totalement. Pour le seed réussi, les centroïdes successifs tracent un chemin lisse et parfaitement ordonné, interprétable d'un seul coup d'œil comme la trajectoire du front de déblocage. Pour le seed effondré, les points se mélangent sans ordre apparent, sans qu'aucun chemin clair ne se dégage. Pour le seed réussi, la dispersion entre fronts F différents est en moyenne 4.05 fois plus grande que la dispersion au sein d'un même front (le bruit de composition de fenêtre). Donc, ϕ résume donc bien le front de déblocage réel plutôt que les salles spécifiques tirées dans la fenêtre. Pour le seed effondré, ce ratio tombe à 1.95. Le signal et le bruit de composition deviennent du même ordre de grandeur, donc ϕ y est une représentation nettement moins fiable de l'état de compétence réel, un agent qui n'apprend pas produit aussi un ϕ moins informatif, pas seulement une politique moins performante.

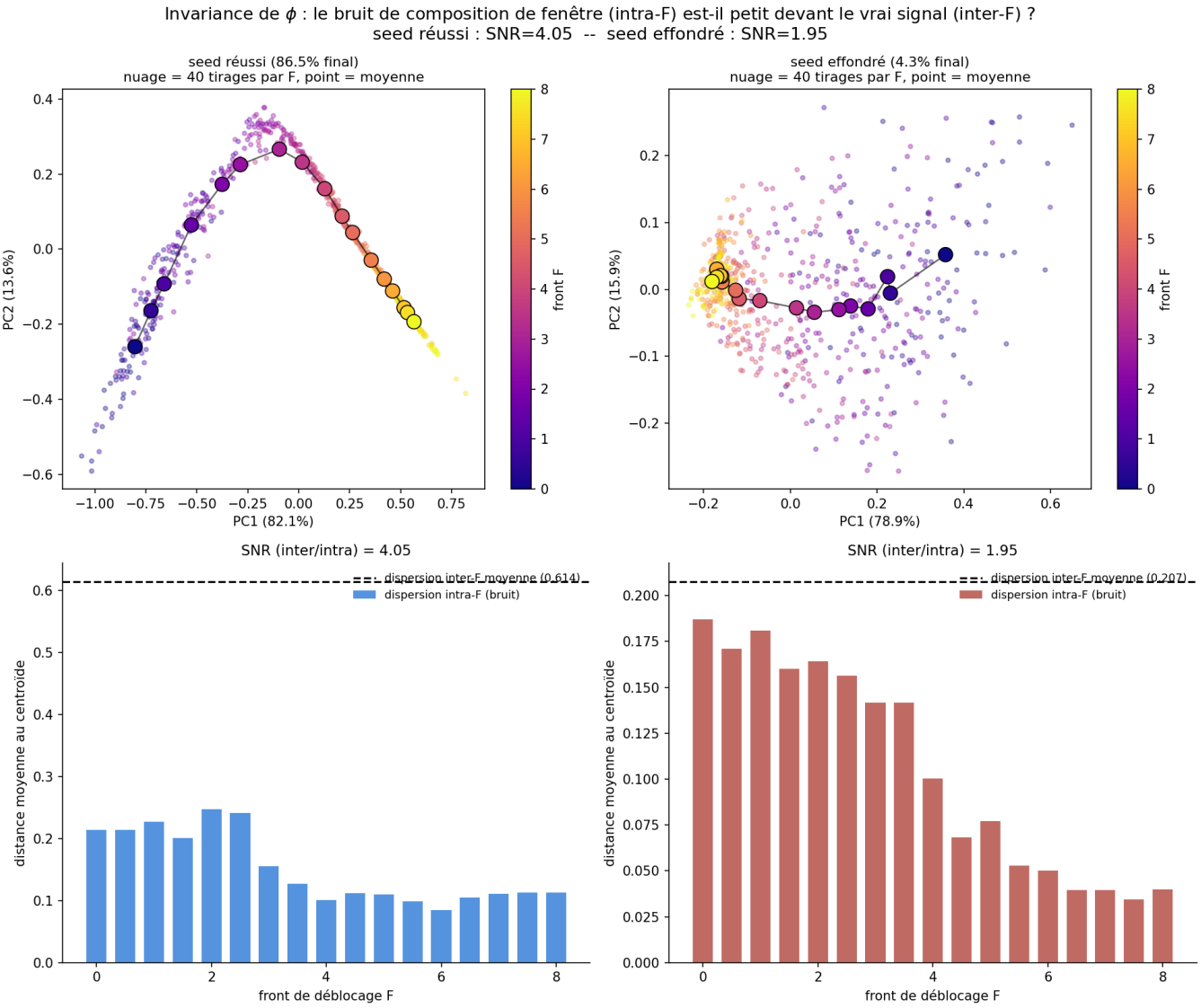
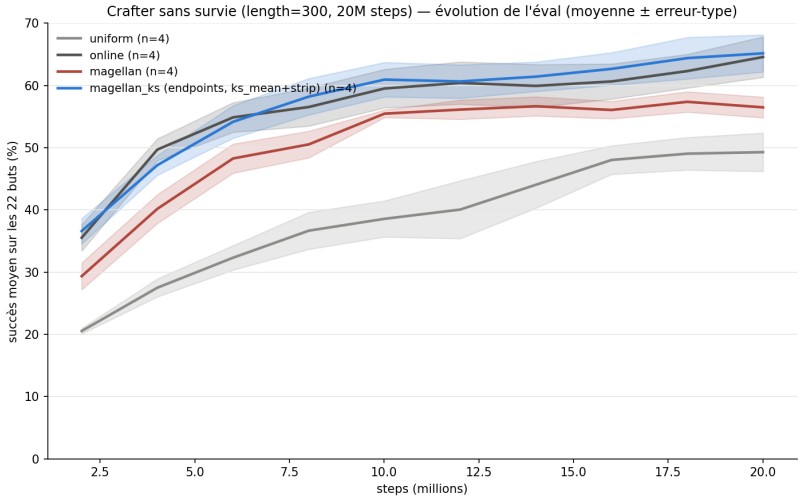


FIGURE 13 – MiniGrid $(5, 5, 8, 8)$, nuages de ϕ et rapport signal/bruit (dispersion inter-F moyenne / dispersion intra-F moyenne), pour un seed réussi (SNR= 4.05) et un seed effondré (SNR= 1.95).

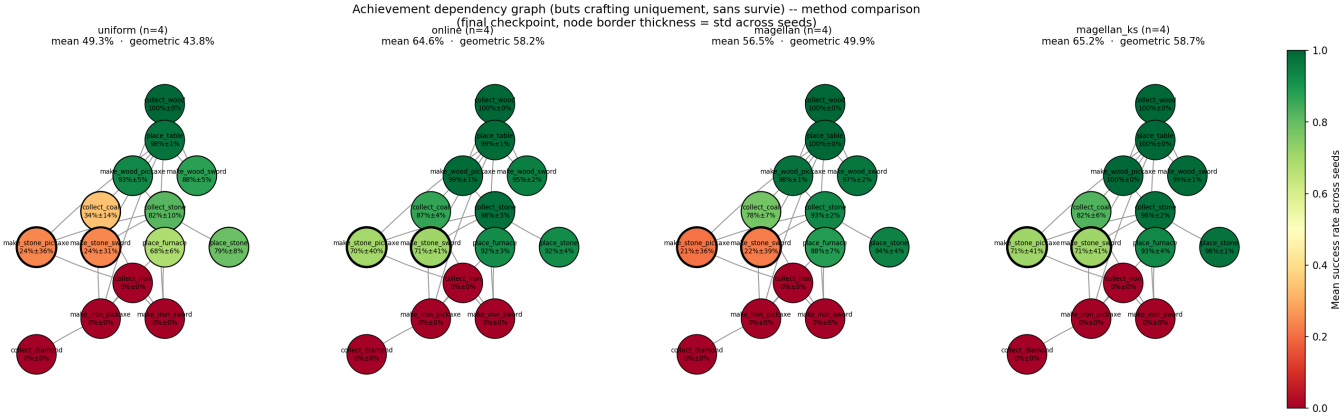
5.3 Crafter

Crafter est un cas nettement plus difficile pour le Reinforcement Learning que MiniGrid car il présente une vraie dépendance combinatoire entre buts, bien plus contraignante que le simple gradient de distance spatiale de MiniGrid, en plus d'un environnement ouvert avec plus d'actions, et d'aléatoire avec la présence de monstres. Avec survie, l'agent meurt pratiquement à chaque épisode s'il ne gère pas activement ses jauges et les menaces. Pour cette raison, on a fixé la longueur d'épisode à 10 000, puisqu'en réalité il meurt quasiment à chaque épisode et l'épisode termine pluts tôt. On présente ici les résultats sans survie (longueur d'épisode 300, les 14 buts de fabrication uniquement) comme condition principale, qui isole cette structure de dépendance entre buts de fabrication, de la difficulté additionnelle liée à la survie. Les résultats avec survie sont qualitativement les mêmes et sont reportés en Annexe F. Dans les deux modes, magellan_ks

utilise sa meilleure configuration connue pour ce jeu, comparée à uniform, online et magellan sur un budget de steps identique entre toutes les méthodes (20M sans survie, 25M avec survie).



(a) Évolution du taux de succès moyen sur les 14 buts de fabrication (moyenne ± erreur-type).



(b) Taux de succès final par but de fabrication, sur l'arbre de dépendance (Annexe C).

FIGURE 14 – Crafter sans survie (longueur 300, 20M steps), *uniform*, *online*, *magellan* et magellan_ks (endpoints + strip). $n = 4$ seeds par courbe.

magellan_ks termine en tête à 65.2% ± 6.9, suivi de près par online (64.6% ± 7.5), magellan restant en retrait (56.5% ± 3.8). Le test de rang checkpoint par checkpoint (magellan_ks contre magellan, H_1 : magellan_ks > magellan, $n = 4$ de chaque côté) atteint une p-valeur de 0.057 à quatre checkpoints différents, avec un écart final non négligeable (+8.7 points), mais avoir plus de seeds donnerait probablement une meilleure puissance statistique.

La Figure 14b décompose le taux de succès final sur l'arbre de dépendance des 14 buts de fabrication, pour voir où les curricula bloquent précisément. Aucune méthode ne dépasse jamais 0.0% sur collect_iron, make_iron_pickaxe, make_iron_sword et collect_diamond, le palier du fer reste un mur structurel infranchi par toutes les méthodes testées, curriculum appris ou non, y compris magellan_ks malgré son avantage sur les paliers précédents. online et magellan_ks réussissent tous deux le palier pierre à un niveau comparable

($\sim 70\% \pm 47$ pour les deux), magellan seul restant nettement en retrait ($\sim 21\% \pm 43$), cohérent avec l'absence de significativité statistique de l'écart magellan_ks/magellan observée plus haut, où c'est surtout magellan, pas online, qui est distancé. Autrement dit, magellan_ks bat nettement magellan et reste compétitif avec online, mais comme toutes les autres méthodes, il ne franchit jamais lui-même la marche vers le fer. Avec survie, le même mur du fer apparaît exactement de la même façon (Figure 21, Annexe F), avec cette fois une différenciation plus nette entre méthodes sur le palier pierre en faveur de magellan_ks : make_stone_pickaxe à 42.5% ± 35.8 et make_stone_sword à 47.9% ± 29.6, contre seulement 8.7% ± 23.4 et 0.8% ± 1.5 pour magellan et 18.8% ± 34.8 / 20.4% ± 36.8 pour online.

Ce mur est-il un problème d'exploration (le curriculum ne consacre jamais assez de pratique au fer) ou une difficulté intrinsèque à la tâche (le fer est trop dur à atteindre pour qu'un signal de récompense binaire et sparse suffise, quel que soit le curriculum) ? Pour trancher entre les deux, on a entraîné les curricula avec un budget doublé (50M au lieu de 25M pas, $n = 4$ seeds) : collect_iron, make_iron_pickaxe, make_iron_sword et collect_diamond restent à 0.0% sur les quatre seeds, exactement comme à 25M. Plus de pas seuls ne suffisent donc pas à franchir le mur. Par curiosité, on a aussi testé un *reward shaping* explicite en donnant une récompense +1 à chaque sous-but du chemin vers le fer effectivement débloquée. Celui-ci débloque bien le fer. Le mur n'est donc pas une limite absolue de la tâche, mais plutôt un problème d'exploration sous récompense sparse. Le reward shaping reste cependant hors du cadre de ce stage, qui porte sur le curriculum et non sur la forme de la récompense elle-même, et n'est donc pas creusé plus avant ici.

Comme pour MiniGrid (Section 5.1), les quatre architectures d'encodeur ρ ont été comparées sur Crafter avec survie.

TABLE 3 – Crafter avec survie, mode ALP omni_ks, taux de succès final par architecture, moyenne ± écart-type.

Architecture	Succès final
mean	56.1% ± 3.8 (n=4)
lstm	55.9% ± 4.2 (n=8)
transformer	56.7% ± 3.0 (n=4)
attention	58.8% ± 3.6 (n=8)

attention arrive numériquement en tête, mais l'écart avec les trois autres architectures (55.9–56.7%) est faible devant les écarts-types qui se chevauchent largement. Contrairement à MiniGrid, où mean se détachait nettement (Section 5.1), aucune architecture ne se différencie clairement sur Crafter. C'est cohérent avec le constat de la sous-section suivante. La structure de l'arbre de compétence de Crafter semble, de façon générale, plus difficile à exploiter pour ρ que la structure spatiale de MiniGrid.

On a aussi tenté, sur Crafter, le même type de sonde que pour MiniGrid (Section 5.2), en suivant cette fois la chaîne principale de fabrication (collect_wood → place_table → make_wood_pickaxe → collect_stone → make_stone_pickaxe → collect_iron → make_iron_pickaxe → collect_diamond) dans son ordre réel de dépendance.

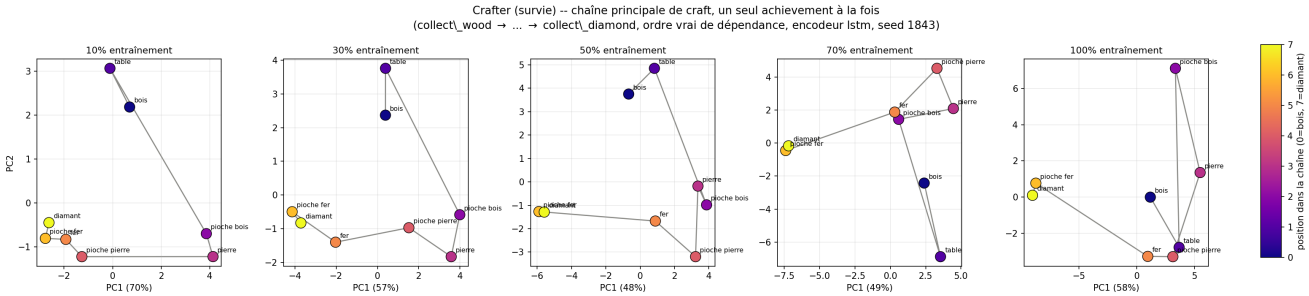


FIGURE 15 – Crafter avec survie, projection ACP des ϕ synthétiques le long de la chaîne principale de fabrication, à 5 moments de l'entraînement. Encodeur lstm.

Contrairement au gradient très lisible obtenu sur MiniGrid (Figure 11), la projection ne trace pas un arc monotone. L'arbre de compétence de Crafter semble donc effectivement plus dur à visualiser proprement que la grille spatiale de MiniGrid, qui a été conçue spécifiquement pour produire ce gradient lisible. L'arbre de dépendance de Crafter combine plusieurs chaînes partiellement indépendantes (fabrication, survie, combat, matériaux) qui ne se réduisent pas aussi naturellement à un unique axe de progression.

5.4 Fetch

L'intérêt de Fetch est qu'il permet de rajouter des buts sans changer la difficulté des buts, et sans changer l'environnement. La Figure 16 étudie le passage à l'échelle du nombre de buts sur Fetch, de 1 000 à 100 000 buts, à budget de steps fixe (20M).

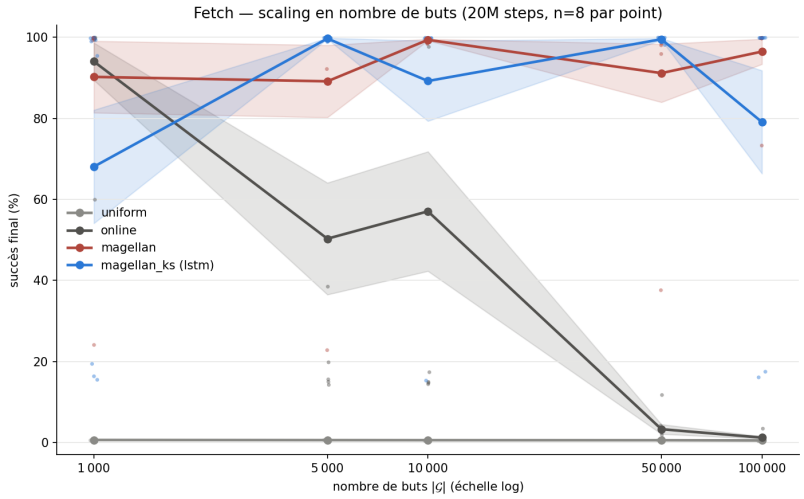


FIGURE 16 – Fetch, taux de succès final en fonction du nombre de buts (échelle log), 20M steps, $n = 8$ par point.

uniform reste au plancher à toute échelle, sans surprise. online s'effondre ici dès 5 000 buts et continue de se dégrader jusqu'à 100 000 (94.0% à 1 000 buts → 1.2% à 100 000) – le passage à l'échelle du nombre de buts fait donc bien décrocher online sur Fetch comme sur MiniGrid 20×20, mais ici dès une échelle beaucoup plus modeste. La raison se lit directement dans le nombre d'épisodes par but. A budget de 20M

pas fixe, on passe de ∼366 épisodes par but à 1 000 buts à ∼1.0 épisode par but seulement à 100 000 buts. online, qui ne généralise jamais entre buts, a mécaniquement besoin de pratiquer chaque but directement pour en estimer l'ALP. magellan et magellan_ks, eux, restent robustes sur toute la plage testée (89–99.5%) précisément parce qu'ils généralisent leur estimation de compétence à des buts peu ou jamais pratiqués directement.

La Figure 17 détaille, à $|\mathcal{G}| = 50\,000$ fixé, le succès final de magellan_ks pour les 4 architectures d'encodeur croisées avec 8 configurations (balanced_training activé/désactivé × correction de fuite aucune/strip/mask_embed, plus prospective_self).

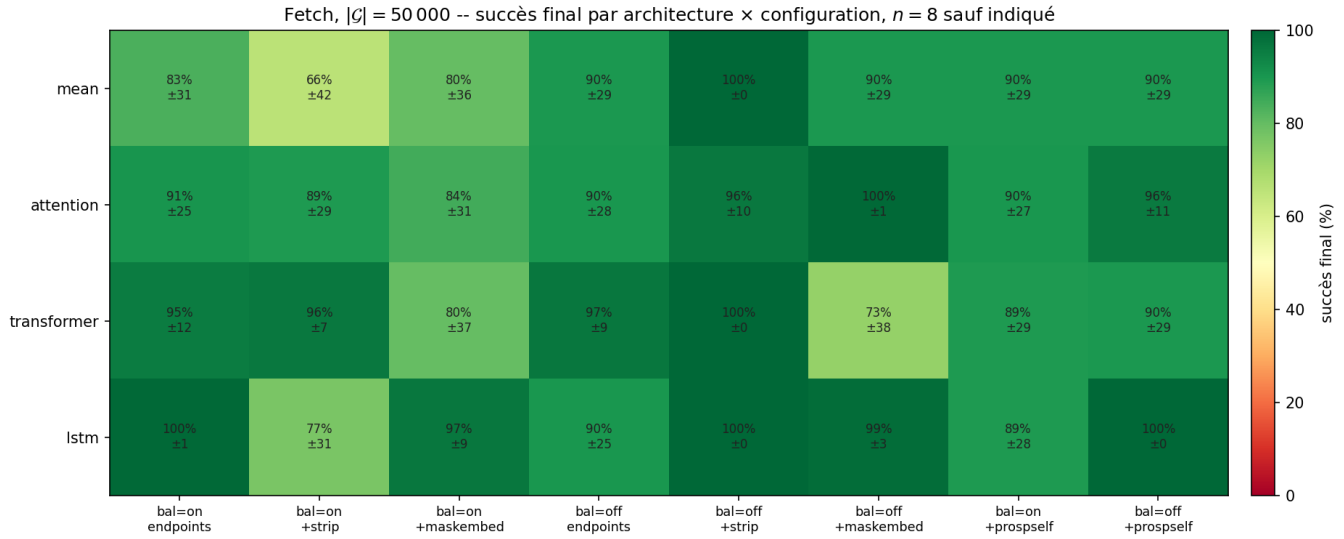


FIGURE 17 – Fetch, 50 000 buts, 20M steps, taux de succès final (moyenne ± écart-type) par architecture × configuration. $n = 8$ sauf indiqué.

Un résultat domine ce grid, en désactivant balanced_training et en activant strip, on atteint 96–100% sur les quatre architectures, avec un écart-type quasi nul. La plupart des autres configurations affichent des écarts-types de 7 à 42 points, signe d'un comportement bimodal. balanced_training semble d'ailleurs nuire plutôt qu'aider à cette échelle. En effet, à correction de fuite identique, le réglage `bal=off` égale ou dépasse systématiquement `bal=on` à l'inverse de ce qu'on observait parfois sur MiniGrid. Ceci est cohérent avec le rôle de ce levier qui cherche à corriger un déséquilibre du buffer entre buts très et peu pratiqués (Section 3.5), un problème surtout présent quand certains buts sont structurellement beaucoup plus faciles ou plus visités que d'autres, comme les salles proches du spawn sur MiniGrid. Sur Fetch, avec 50 000 buts a priori de difficulté comparable, aucun sous-ensemble de buts n'est structurellement sur-représenté dans le buffer et balanced_training n'a donc pas de déséquilibre réel à corriger. Il ne fait qu'ajouter du bruit au gradient en re-pondérant un buffer qui n'en avait pas besoin.

L'espace de buts de Fetch étant continu (Section 4), le pool de buts effectivement utilisé pendant l'entraînement (taille $|\mathcal{G}|$ fixée en amont) ne couvre jamais qu'une sous-partie de l'espace réellement atteignable. On teste donc directement si ϕ généralise à des buts que le curriculum n'a jamais pu échantillonner. On teste donc, sur le run 50 000 buts le plus performant (99.8% final, transformer, bal_off+strip), à un checkpoint

choisi spécifiquement pour avoir un mélange réel de buts maîtrisés et non maîtrisés (checkpoint à 20% d'entraînement : 33% de succès moyen sur l'échantillon d'évaluation), la compétence prédite à partir de la fenêtre ϕ_t réelle de ce moment précis (pas une fenêtre synthétique) pour 50 buts jamais vus à l'entraînement, comparée à leur taux de succès réel mesuré sur 20 épisodes réellement joués avec la politique de ce même checkpoint.

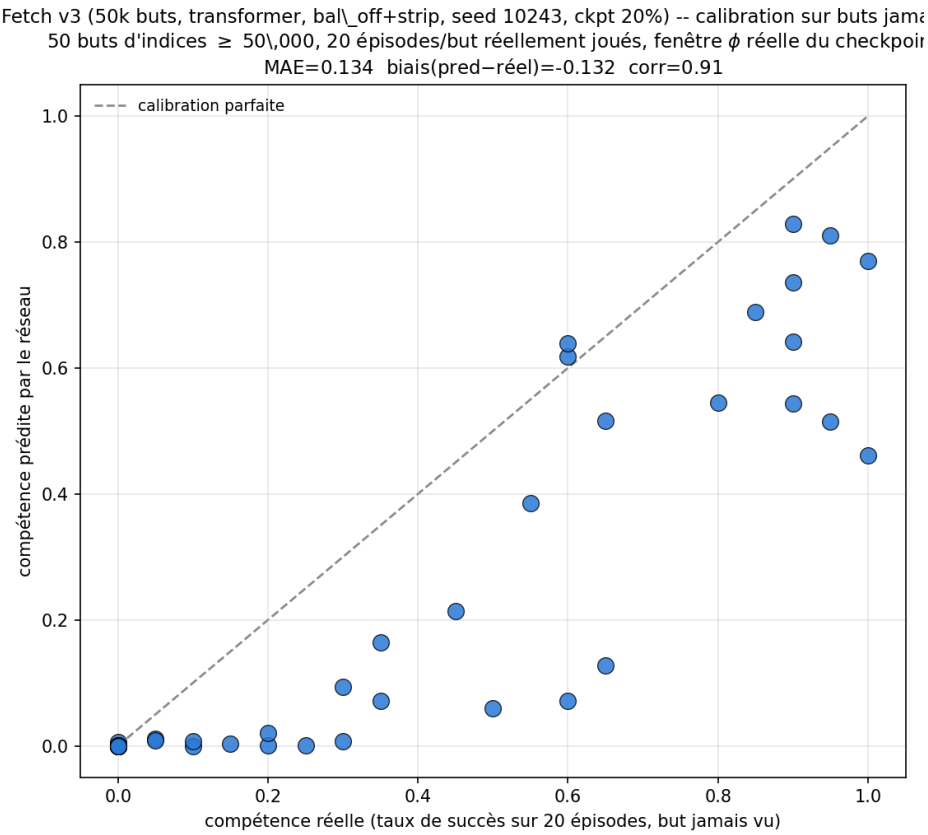


FIGURE 18 – Fetch, 50 000 buts, transformer, bal_off+strip, checkpoint à 20% d'entraînement, compétence prédite vs compétence réelle pour 50 buts jamais vus à l'entraînement.

La calibration est bonne dans l'ensemble avec une MAE = 0.134 et une corrélation de 0.91 entre compétence prédite et compétence réelle. Le réseau ordonne donc correctement les buts faciles et difficiles même sans jamais les avoir rencontrés. On retrouve toutefois un biais systématique −0.132 en moyenne, c'est-à-dire que le réseau sous-estime la compétence réelle plus souvent qu'il ne la surestime, surtout dans le haut de la plage (plusieurs buts à 90–100% de réussite réelle prédits entre 46% et 83%). Une hypothèse pour ce biais est que le réseau n'a jamais reçu de signal d'entraînement associant un but spécifiquement jamais pratiqué à une compétence très élevée. Tout son signal de succès quasi certain provient de buts réellement observés comme maîtrisés dans le buffer $\mathcal{M}$ et se comporte donc avec une forme de prudence face à un but qu'il n'a, par construction, jamais pu confirmer directement.

6 Conclusion

Ce stage partait d'un constat précis sur MAGELLAN. Son estimateur de compétence est en fait une cible mouvante et donc ne peut comparer directement deux évaluations de compétence prises à des instants différents. La proposition de ce stage était d'ajouter un conditionnement sur un état de connaissance ϕ_t, résumé vectoriel de l'historique récent de l'agent, dans l'espoir de rendre la relation $(g, \phi) \mapsto$ compétence stationnaire au cours de l'entraînement.

Au long de ce stage, j'ai réalisé quatre architectures d'agrégation (mean, attention, lstm, transformer), implémenté plusieurs mécanismes de calcul du *learning progress* à partir de ϕ. Je les ai comparé et validé sur trois environnements construits spécifiquement pour ce stage, chacun sollicitant une structure différente de l'espace des buts. magellan_ks bat MAGELLAN sur MiniGrid aux deux échelles testées, reste compétitif ou meilleur qu'online et MAGELLAN sur Crafter malgré le mur structurel du fer, et passe à l'échelle sur Fetch jusqu'à 100 000 buts là où online s'effondre. Au-delà de la seule performance du curriculum, plusieurs sondes construites pendant ce stage ont permis de vérifier directement, au niveau de la représentation ϕ elle-même, que l'état de connaissance encode une structure cohérente avec la difficulté réelle des buts sur MiniGrid, et qu'il généralise avec un biais conservateur plutôt qu'optimiste sur des buts jamais pratiqués sur Fetch.

Plusieurs éléments n'ont pas pu être réalisés, ou sont restés incomplets. Le mur du fer sur Crafter n'a jamais été franchi par aucune méthode de curriculum testée, y compris en doublant le budget d'entraînement. Seul un reward shaping explicite, hors du cadre de ce stage, a permis de le débloquer. Cela situe cette limite du côté de la récompense sparse plutôt que du curriculum lui-même. La structure de ϕ s'est aussi avérée nettement plus difficile à visualiser proprement sur Crafter que sur MiniGrid. La grille spatiale de MiniGrid avait justement été conçue pour offrir un gradient de difficulté lisible selon un seul axe, alors que l'arbre de dépendance de Crafter, avec ses chaînes partiellement indépendantes, ne s'y prête pas aussi naturellement. Enfin, certaines questions ouvertes n'ont pas pu être tranchées faute de temps. On ne sait pas pourquoi l'effet du sweep sur la taille de fenêtre K contredit l'intuition initiale pour les trois architectures testées. On ne sait pas non plus pourquoi la correction de fuite d'auto-occurrence change si peu la performance d'attention, alors qu'elle exploite activement ce raccourci comme transformer.

Plusieurs directions restent ouvertes pour poursuivre ce travail. Il faudrait d'abord disposer de davantage de seeds pour renforcer la puissance statistique des comparaisons. Plusieurs écarts rapportés dans ce rapport restent proches du seuil de significativité sans le franchir, faute d'un échantillon suffisant. Il serait aussi utile d'étudier directement comment éviter l'effondrement observé pour certaines configurations sur MiniGrid. Un autre axe consisterait à comprendre plus précisément l'origine du biais prudent observé dans l'estimation de compétence sur des buts jamais vus sur Fetch, et à chercher comment le corriger, par exemple en recalibrant la sortie du réseau ou en modifiant la perte d'entraînement. Enfin, la mesure prospective_self ne simule aujourd'hui qu'un seul pas hypothétique en avant. Il serait intéressant d'étudier si une version prospective à plusieurs pas apporte un signal de progrès encore meilleur, malgré le coût de calcul supplémentaire que cela impliquerait.

Références

Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. (2020). A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 1597–1607. PMLR.

Chevalier-Boisvert, M., Dai, B., Towers, M., de Lazcano, R., Willems, L., Lahlou, S., Pal, S., Castro, P. S., and Terry, J. (2023). Minigrid & miniworld : Modular & customizable reinforcement learning environments for goal-oriented tasks. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*.

Colas, C., Fournier, P., Chetouani, M., Sigaud, O., and Oudeyer, P.-Y. (2019). CURIOUS : Intrinsically motivated modular multi-goal reinforcement learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 1331–1340. PMLR.

de Lazcano, R., Andreas, K., Tai, J. J., Lee, S. R., and Terry, J. (2024). Gymnasium robotics. http ://github.com/Farama-Foundation/Gymnasium-Robotics.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT : Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics : Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Gaven, L., Carta, T., Romac, C., Colas, C., Lamprier, S., Sigaud, O., and Oudeyer, P.-Y. (2025). Magellan : Metacognitive predictions of learning progress guide autotelic llm agents in large goal spaces. In *International Conference on Machine Learning (ICML)*.

Hafner, D. (2022). Benchmarking the spectrum of agent capabilities. In *10th International Conference on Learning Representations (ICLR)*.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8) :1735–1780.

Kaplan, F. and Oudeyer, P.-Y. (2007). In search of the neural circuits of intrinsic motivation. *Frontiers in Neuroscience*, 1(1) :225–236.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540) :529–533.

Oudeyer, P.-Y., Kaplan, F., and Hafner, V. V. (2007). Intrinsic motivation systems for autonomous mental development. *IEEE Transactions on Evolutionary Computation*, 11(2) :265–286.

Pathak, D., Agrawal, P., Efros, A. A., and Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2778–2787. PMLR.

Plappert, M., Andrychowicz, M., Ray, A., McGrew, B., Baker, B., Powell, G., Schneider, J., Tobin, J., Chociej, M., Welinder, P., Kumar, V., and Zaremba, W. (2018). Multi-goal reinforcement learning : Challenging robotics environments and request for research. *arXiv preprint arXiv :1802.09464*.

Portelas, R., Colas, C., Hofmann, K., and Oudeyer, P.-Y. (2020). Teacher algorithms for curriculum learning of deep rl in continuously parameterized environments. In *Proceedings of the 3rd Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 835–853. PMLR.

Puterman, M. L. (1994). *Markov Decision Processes : Discrete Stochastic Dynamic Programming*. John Wiley & Sons.

Schulman, J., Moritz, P., Levine, S., Jordan, M. I., and Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. In *4th International Conference on Learning Representations (ICLR)*.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv :1707.06347*.

Sorg, J., Singh, S., and Lewis, R. L. (2010). Variance-based rewards for approximate bayesian reinforcement learning. In *Proceedings of the 26th Conference on Uncertainty in Artificial Intelligence (UAI)*, volume R8 of *Proceedings of Machine Learning Research*, pages 563–570. PMLR.

Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems 12 (NIPS 1999)*, pages 1057–1063.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 5998–6008.

Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R., and Smola, A. (2017). Deep sets. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 3391–3401.

Zhang, J., Lehman, J., Stanley, K., and Clune, J. (2024). OMNI : Open-endedness via models of human notions of interestingness. In *12th International Conference on Learning Representations (ICLR)*.

A Liste des hyperparamètres

Paramètre	Rôle	Défaut
Entraînement PPO (Section 2.2)		
`lr`	Taux d'apprentissage (acteur-critique)	3×10^{-4}
`gamma` (γ)	Facteur d'actualisation	0.99
`lam` (λ)	Paramètre de GAE	0.95
`clip_eps` (ϵ)	Amplitude de clipping de L^{CLIP} (éq. (4))	0.2
`ppo_epochs`	Nombre d'époques de descente de gradient par rollout collecté	4
`entropy_coef` (c_2)	Poids du bonus d'entropie (éq. (7))	0.01
`rollout_len`	Longueur d'un rollout (en pas par environnement)	1024
`n_envs`	Nombre d'environnements parallèles	32
`num_minibatches`	Nombre de mini-batchs par rollout	4
`enc_dim`	Dimension de l'encodeur d'observation	128
`hidden_dim`	Dimension cachée de la tête acteur-critique	512
Fenêtre et buffers		
`ks_window` (K)	Taille de la fenêtre glissante w_t	30
`ks_train_buffer` (M)	Taille du buffer d'entraînement $\mathcal{M}$ du MLP de compétence	50 000
Entraînement du MLP de compétence		
`magellan_train_every` (κ)	Cadence de ré-entraînement (en épisodes)	32
`ks_train_batch` (B)	Taille du mini-batch $\mathcal{B} \subset \mathcal{M}$ tiré à chaque pas	256
k	Nombre de pas de descente de gradient par ré-entraînement	20
`ks_balance_alpha` (α_{bal})	Force de la correction d'équilibrage (0 = équilibrage total, 1 = aucune correction)	0.0
`main_leave_self_out`	Active la correction de fuite d'information sur la perte principale	désactivé
`main_leave_self_out_mode`	strip / mask_embed	strip
Calcul de l'ALP (Section 3.3)		
`magellan_snapshots` (N)	Taille du buffer de snapshots $\mathcal{N}$ pour l'ALP rétrospective	10
`ks_alp_mode`	endpoints / to_now / omni_ks / prospective_self / prospective_global	endpoints
`ks_alp_ema` (α_{ema})	Constante de lissage EMA (mode *omni_ks*)	–
`ks_alp_p_theta` (θ)	Paramètre de repondération f_θ (mode *omni_ks*)	0.1
`ks_prospective_subsample` ($\lvert\mathcal{S}\rvert$)	Taille du sous-échantillon $\mathcal{S}$ (mode *prospective_global*)	50

Paramètre	Rôle	Défaut
Architecture de l'encodeur (ρ, Section 3.1 et suivante)		
`ks_item_hidden` (d_h)	Dimension cachée de ψ et de la sortie des étapes d'agrégation	32
`ks_dim` (d_ϕ)	Dimension de l'état de connaissance $\phi_t \in \Phi$	16
`ks_pool_mode`	mean / attention / lstm / transformer	mean
`ks_pool_heads`	Nombre de têtes (modes *attention*/*transformer*)	1
`ks_transformer_layers`	Nombre de couches d'auto-attention (modes *transformer*)	2

B Mécanismes additionnels non retenus

Deux mécanismes supplémentaires ont été implémentés et testés en complément de la perte principale $\mathcal{L}$ (Section 3.5), sans résultat suffisamment net pour être retenus dans la configuration finale.

Perte auxiliaire de décodabilité En plus de prédire la compétence sur le but de l'exemple courant, on force ϕ à rester décodable en taux de succès pour tous les autres buts présents dans la même fenêtre, au moyen d'une tête linéaire supplémentaire $D : \Phi \to \mathbb{R}^{|\mathcal{G}|}$:

$$\mathcal{L}_{\text{aux}} = -\frac{1}{|\mathcal{B}'|} \sum_{(g,w,r)\in\mathcal{B}'} \frac{1}{|\{g' \in w\}|} \sum_{g'\in w} \Big[\bar{r}_{g'} \log \sigma\big(D(\rho(w))_{g'}\big) + (1-\bar{r}_{g'}) \log\big(1-\sigma(D(\rho(w))_{g'})\big)\Big],$$

où $\bar{r}_{g'}$ est le taux de succès récent du but g' dans la fenêtre w, et la somme ne porte que sur les buts g' effectivement présents dans w, les autres composantes de sortie de D sont ignorées par un masquage. La perte totale devient $\mathcal{L} + \lambda_{\text{aux}}\mathcal{L}_{\text{aux}}$.

Perte contrastive (NT-Xent) L'idée, inspirée de SimCLR (Chen et al., 2020), est de rapprocher deux vues différentes d'une même fenêtre dans l'espace Φ, et éloigner les fenêtres associées à des exemples différents du même mini-batch. Pour chaque exemple $(g, w, r) \in \mathcal{B}'$, on tire deux sous-échantillons indépendants de w, chacun conservant une fraction des items, puis on les encode séparément : $z_1 = \rho(w^{(1)})$, $z_2 = \rho(w^{(2)})$. La perte NT-Xent standard est appliquée directement sur ces vecteurs :

$$\mathcal{L}_{\text{cl}} = -\frac{1}{2B}\sum_{i=1}^{2B} \log \frac{\exp(\text{sim}(z_i, z_{j(i)})/\tau)}{\sum_{k\neq i}\exp(\text{sim}(z_i,z_k)/\tau)}, \qquad \text{sim}(u,v) = \frac{u\cdot v}{\|u\|\,\|v\|},$$

où B est la taille du batch, $j(i)$ désigne l'indice de l'autre vue du même exemple que i (l'unique paire positive de i), τ la température, et la somme au dénominateur court sur les $2B-1$ autres vecteurs du batch (positif et négatifs confondus, sauf i lui-même).

C Encodage des buts Crafter par arbre de dépendance

Plutôt qu'un *one-hot encoding* ou une représentation apprise par LLM, chaque but Crafter est encodé par un vecteur fixe et construit à la main, qui encode explicitement sa position dans l'arbre de dépendance du jeu plutôt que de laisser un modèle la redécouvrir à partir des données d'entraînement seules.

Un graphe orienté acyclique de 25 arêtes encode les véritables prérequis de fabrication de Crafter. Pour un but g, on calcule la fermeture transitive de ses prérequis directs, pas seulement ses parents immédiats, mais tous les ancêtres du graphe, notée $\mathrm{Pre}(g)$. Trois buts (`eat_cow`, `defeat_zombie`, `defeat_skeleton`) ne nécessitent aucune épée, mais en bénéficient puisque toute épée augmente les dégâts au corps à corps. Cette relation d'aide, pas de prérequis strict est appelée dépendance molle.

Pour un but g, sur un espace de $n = 22$ buts, le vecteur $v(g) \in \mathbb{R}^{n+A+O}$ ($A = 6$ types d'action, $O = 6$ types d'objet/matériau, soit 34 dimensions) se décompose en trois blocs :

— n **premières composantes** ($v_{g'}$, une par but g' de l'espace des buts, lui-même inclus) : code la relation entre g et g'

$$v_{g'}(g) = \begin{cases} 3 & \text{si } g' = g \quad \text{(identité)} \\ 2 & \text{si } g' \in \mathrm{Pre}(g) \quad \text{(prérequis dur)} \\ 1 & \text{si } g \in \{\texttt{eat_cow}, \texttt{defeat_zombie}, \texttt{defeat_skeleton}\} \text{ et } g' \text{ est une épée} \\ 0 & \text{sinon.} \end{cases}$$

— $A = 6$ **composantes suivantes** : encodage one-hot du *type d'action* de g
`collect/place/make/eat/defeat/wake`

— $O = 6$ **dernières composantes** : encodage multi-hot du ou des *matériaux* impliqués dans g `wood/stone/iron/sword/pickaxe/other`, extraits directement du nom de l'achievement.

D Configurations alternatives de magellan_ks

Le corps du rapport présente, pour chaque environnement, les résultats de la meilleure configuration connue de magellan_ks. Cette annexe rassemble les comparaisons systématiques entre configurations alternatives (architecture d'encodeur, correction de fuite, ALP, balanced_training) qui ont permis d'identifier ces configurations, mais dont le détail alourdirait inutilement le corps du texte.

Le Tableau 5 étend, à l'échelle MiniGrid 20×20 (Section 5.1), la comparaison à l'ensemble des configurations magellan_ks testées à cette échelle.

TABLE 5 – MiniGrid 20×20 (9999 buts, 24M steps) – taux de succès final, moyenne ± écart-type, $n = 8$ seeds par ligne.

Configuration	Succès final
uniform	0.05% ± 0.08
online	0.21% ± 0.13
magellan	1.36% ± 0.37
ks_mean, endpoints (balanced_training activé)	1.64% ± 0.27
ks_mean, +strip (balanced_training activé)	1.41% ± 0.31
ks_mean, +prospective_self (balanced_training activé)	**1.90**% ± 0.93
ks_mean, balanced_training désactivé	1.48% ± 0.44
ks_mean, bestcombo (balanced_training désactivé + prospective_self + strip)	1.51% ± 0.66
ks_mean, +prospective_self +strip (balanced_training activé)	1.43% ± 0.42
ks_lstm, endpoints (balanced_training activé)	1.86% ± 0.37
ks_attention, endpoints (balanced_training activé)	1.83% ± 0.51
ks_attention, +prospective_self (balanced_training activé)	1.40% ± 0.63

E Sonde de sensibilité à l'auto-occurrence

Pour vérifier directement, au niveau de la représentation, que *main_leave_self_out* réduit bien la dépendance de $C(g, \phi)$ à l'auto-occurrence de g dans sa propre fenêtre (et pas seulement d'en constater l'effet sur la performance finale, Section 5), la sonde suivante a été construite. Pour un checkpoint donné et $n = 40$ buts candidats tirés aléatoirement (seed fixée, mêmes buts pour toutes les conditions comparées) :

1. Calculer $C(g, \phi)$ avec la fenêtre réelle telle quelle, C_{sans}.
2. Remplacer les 3 entrées les plus anciennes de la fenêtre par 3 occurrences synthétiques $(g, 1)$ (succès), ré-encoder, et recalculer, C_{avec}.
3. La sensibilité à l'auto-occurrence pour ce but est $|C_{\text{avec}} - C_{\text{sans}}|$.

L'injection forcée de 3 occurrences synthétiques (plutôt que de compter sur des répétitions naturelles du but dans la fenêtre, qui donnent un nombre d'occurrences très variable d'une condition à l'autre et rendent la comparaison statistiquement fragile) garantit une comparaison contrôlée, à $n = 40$ fixé, entre conditions. Sur MiniGrid $(5, 5, 8, 8)$, pooling *mean*, à convergence :

Condition	**Sensibilité moyenne**
baseline (pas de correction)	0.0254
+ strip	0.0107
+ mask	0.0125
+ mask_embed	0.0121
+ prospective_self	0.0100

Les quatre mécanismes réduisent la sensibilité mesurée d'environ moitié par rapport à la baseline, dans des proportions comparables entre eux.

Une seconde sonde, ciblée cette fois sur attention et transformer, teste directement si leur mécanisme de pondération appris sous-pondère spontanément l'item en autoréférence (hypothèse évoquée Section 5.1). Sur les checkpoints *baseline* (sans correction de fuite) de chaque architecture, on construit 300 fenêtres synthétiques contenant chacune un item en autoréférence (même but que celui interrogé, succès= 1) parmi $K-1$ autres items aléatoires, et on mesure le poids réellement reçu par cet item, α_i pour attention (exposé par `attention_weights()`), et l'attention reçue par la première couche d'auto-attention pour transformer (pas de poids de pooling unique disponible pour ce dernier, la réduction finale étant une moyenne masquée).

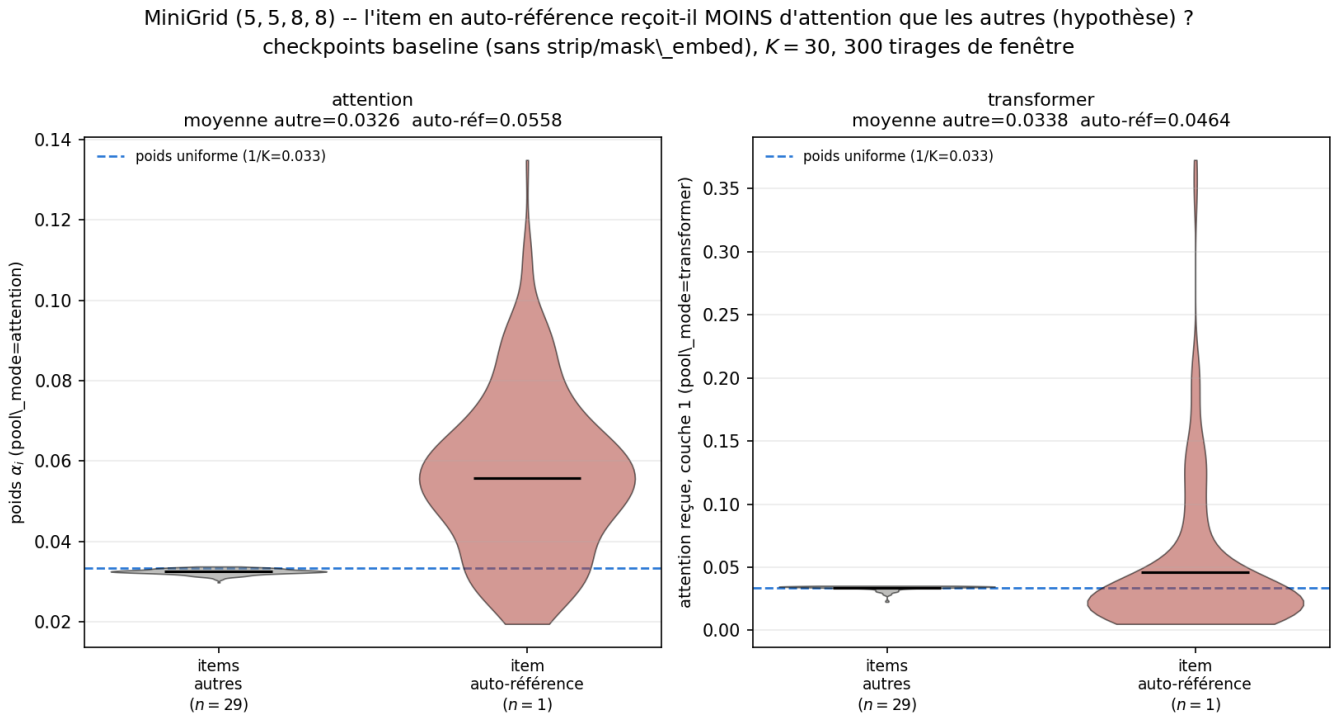


FIGURE 19 – MiniGrid $(5, 5, 8, 8)$, checkpoints baseline (sans correction de fuite) – poids reçu par l'item en autoréférence contre les $K-1$ autres items de la fenêtre, pour attention et transformer. 300 tirages de fenêtre, $K = 30$.

Le résultat contredit l'hypothèse. L'item en autoréférence reçoit en réalité *plus* de poids que les autres, pas moins, pour les deux architectures. Pour attention, il reçoit en moyenne 0.056 contre 0.033 pour un item quelconque (soit très exactement le poids uniforme $1/K$) : les autres items sont traités de façon presque parfaitement égale, et c'est l'item en autoréférence qui capte l'excédent de poids. Le même excès s'observe pour transformer (0.046 contre 0.034), avec une dispersion nettement plus grande (certains tirages atteignant 0.3+), cohérent avec le fait que son architecture laisse les items interagir sur plusieurs couches avant la réduction finale. Le mécanisme de pondération apprise ne protège donc spontanément ni attention ni transformer de la fuite d'auto-occurrence : les deux s'y appuient activement plutôt que de l'éviter. Pour transformer, cette sonde apporte donc une preuve directe, au niveau de la représentation, que la correction *strip* est justifiée et pas seulement empiriquement bénéfique. Le modèle non corrigé exploite réellement

ce raccourci, ce qui explique l'écart de performance considérable observé entre la version corrigée et non corrigée (26.8% → 62.4%, Figure 9). Pour attention, en revanche, le même excès de poids est mesuré, mais sans qu'il se traduise par un gain de performance comparable une fois corrigé (46.5% → 47.8%). La fuite existe bien, mais son impact réel sur la performance reste une question ouverte, peut-être parce qu'attention dispose, par ailleurs, d'assez de capacité de généralisation sur les buts non auto-référents pour compenser ce raccourci plutôt que d'en dépendre entièrement.

F Crafter avec survie

Cette annexe complète la Section 5.3 avec les résultats Crafter **avec survie** (longueur d'épisode 10 000, les 22 buts), qualitativement identiques à ceux obtenus sans survie : magellan_ks reste en tête, et le mur du fer bloque toutes les méthodes de la même façon.

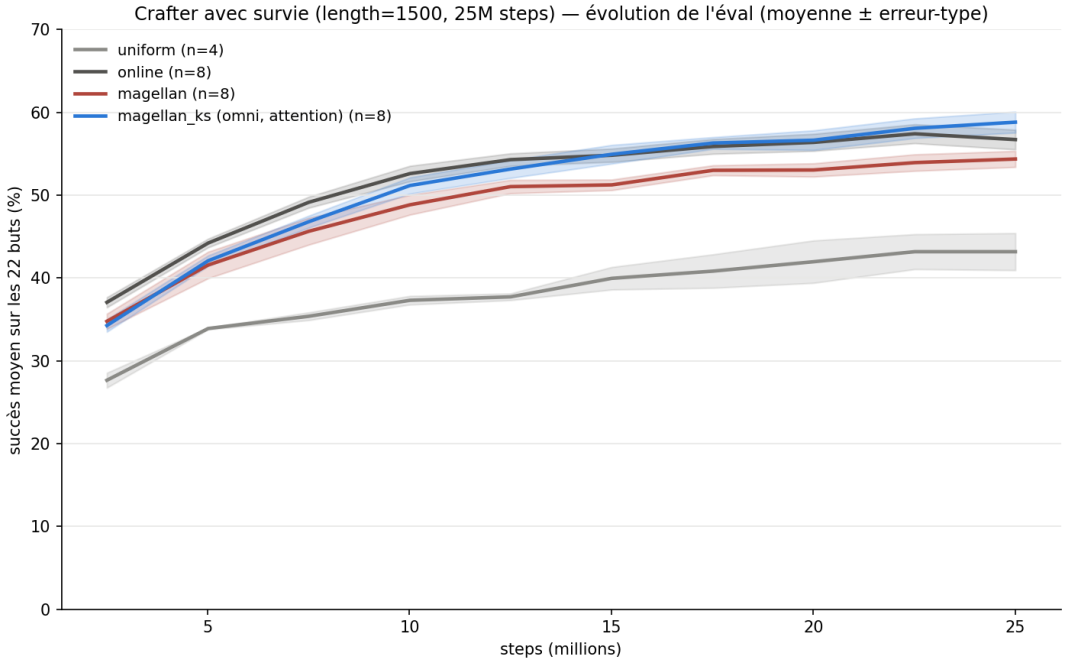


FIGURE 20 – Crafter avec survie (longueur 10 000, 25M steps), évolution du taux de succès moyen sur les 22 buts (moyenne ± erreur-type).

magellan_ks termine à 58.8% ± 3.8 contre 56.7% ± 3.6 pour online, 54.4% ± 2.9 pour magellan et 43.2% ± 5.2 pour uniform. Le test de rang checkpoint par checkpoint (magellan_ks contre magellan, H_1 : magellan_ks > magellan, $n = 8$ de chaque côté) est non significatif en début d'entraînement puis devient significatif à partir de 15M steps et le reste jusqu'à la fin (p entre 0.0074 et 0.019 sur les 5 derniers des 10 checkpoints).

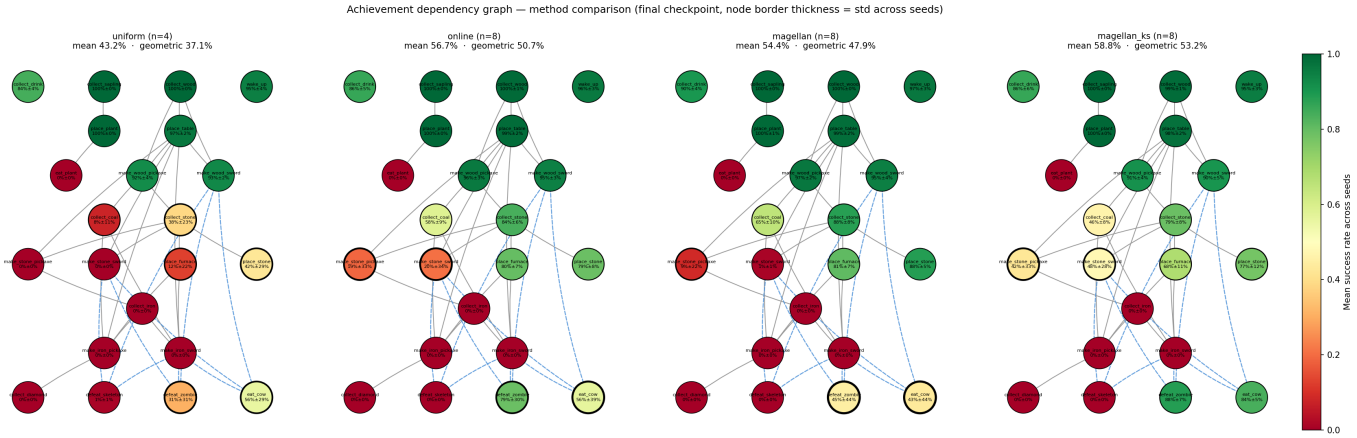


FIGURE 21 – Crafter avec survie, taux de succès final par but, sur l'arbre de dépendance des achievements. Épaisseur de bordure = écart-type inter-seeds.